



Saad Dahlab University - Blida1
Faculty of Sciences
Department of Computer Science
Computer Science Engineering
Semester 1 (1st year)



ALGORITHMIC AND DATA STRUCTURES 1

Dr. MUSTAPHA CHABA MOUNA

(chabamouna_mustapha@univ-blida.dz)

2025-2026

PREAMBLE

❖ Weekly Hourly Volume

Unités d'Enseignement	Intitulés des matières	Crédits	Coef.	V.H hebdomadaire			VHS	Mode d'évaluation	
				C	TD	TP	15 sem.	Continu	Examen
UE Fondamentale Code : UEF 1.1.1 Crédits : 12 Coefficients : 12	Algorithmique et structure de données 1	6	6	3h00	1h30	1h30	90h	40%	60%
	Analyse mathématique 1	6	6	3h00	3h00		90h	40%	60%
UE Fondamentale Code : UEF 1.1.2 Crédits : 13 Coefficients : 12	Architecture des ordinateurs 1	5	4	3h00	1h30		67h30	40%	60%
	Introduction aux systèmes d'exploitation 1	5	4	1h30		3h00	67h30	40%	60%
	Algèbre 1	3	4	1h30	1h30		45h	40%	60%
UE Méthodologique Code : UEM 1.1 Crédits : 2 Coefficients : 1	Technique d'expression écrite	2	1	1h30			22h30		100%
UE Découverte Code : UED 1.1 Crédits : 3 Coefficients : 1	Bureautique et Web	3	1	1h30		1h30	45h	40%	60%
Volume Horaire Total		30	26	15h	7h30	6h	427h30		

[According to the annex of decree No. 060 of January 15, 2023, which sets the curriculum for the common core program leading to the State Engineer diploma in the field of "Mathematics and Computer Science"; major: "Computer Science."]

PREREQUISITES

❖ What do I need to master algorithmic? (Required skills)?

- A minimum of general culture, as the problems to solve come from various fields.
- Understanding the language used in the problem statement (French and/or English).
- Mastery of some mathematical concepts.
- Be rigorous, have good logic, and a lot of patience and courage.

❖ What should I do to master algorithmic?

- Love the module to find your own solution to the given problem.
- Regularly and continuously review the module (at least once a week).
- Learn the method of solving rather than the solution itself.
- Do a lot of exercises to improve reflexes.

OBJECTIVES

- ❖ Acquisition of a **methodological approach** focused on **modularity** that allows the **design** and **realization** of a small software using elementary **objects** and **static data structures** (one-dimensional and two-dimensional arrays, strings, sets, records).
- ❖ Be able to:
 - Proceed with the modular decomposition of the software.
 - Analyze and separately build the different modules (main and secondary).
 - Validate each module.
 - Make a technical programming report.

CONTENT

- I. BASIC ELEMENTS**
- II. PRESENTATION OF ALGORITHMIC FORMALISM**
- III. BASIC ELEMENTS OF C LANGUAGE**
- IV. ARRAYS & STRING**
- V. PERSONLISED TYPES**



Saad Dahlab University - Blida1

Faculty of Sciences

Department of Computer Science

Computer Science Engineering

Semester 1 (1st year)

Algorithmic and Data Structures 1



CHAPTER I: BASIC ELEMENTS

Dr. MUSTAPHA CHABA MOUNA

(chabamouna_mustapha@univ-blida.dz)

2025-2026

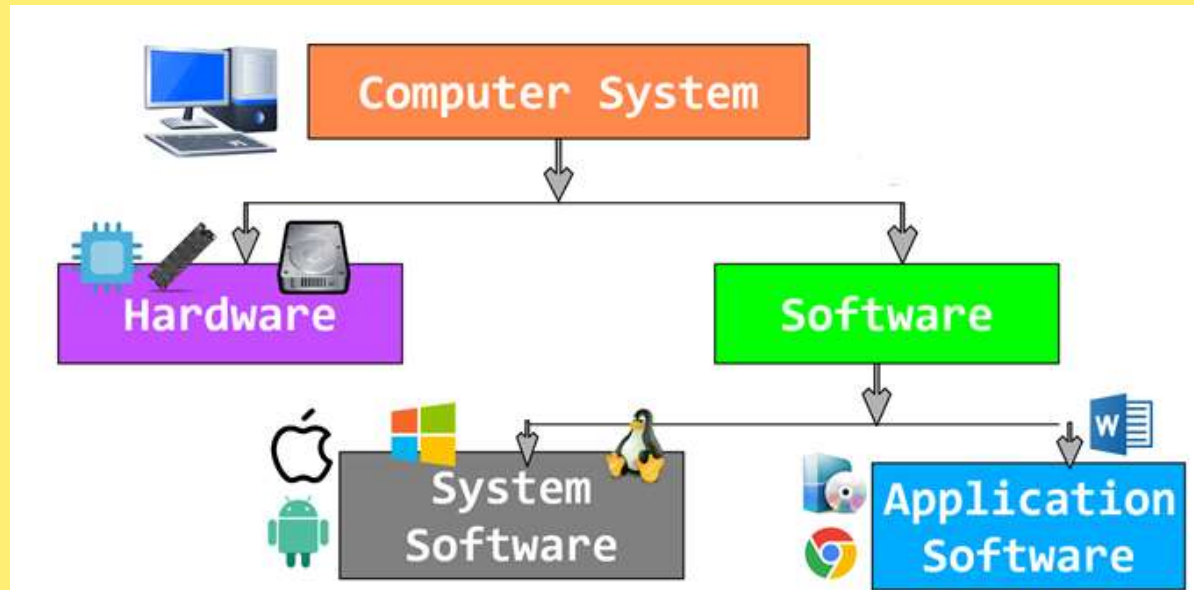
CONTENT

- ❑ Basic Concepts: Computer Science, Algorithm, Action, Processor, Computer, etc.
- ❑ Programming: Programs and Programming Languages
- ❑ Problem Solving (Analysis and step-by-step Process)

BASIC CONCEPTS

❖ Definitions 1:

- ❖ Computer science is the study of **computers** and computational systems. It covers (includes) the theory, design, development, and application of the **software** and **hardware** components that make up these systems.



BASIC CONCEPTS

INPUT DEVICE



MOUSE



KEYBOARD



SCANNER



JOYSTICK



WEB CAMER

OUTPUT DEVICE



MONITOR



SPEAKER



PRINTER



PROJECTOR



HEADPHONE

COMPUTER



PROCESSING DEVICE



CPU
(CENTRAL PROCESSING UNIT)

STORAGE DEVICE



PENDRIVE



HARD DISK



MEMORY CARD



FLOPPY DISK

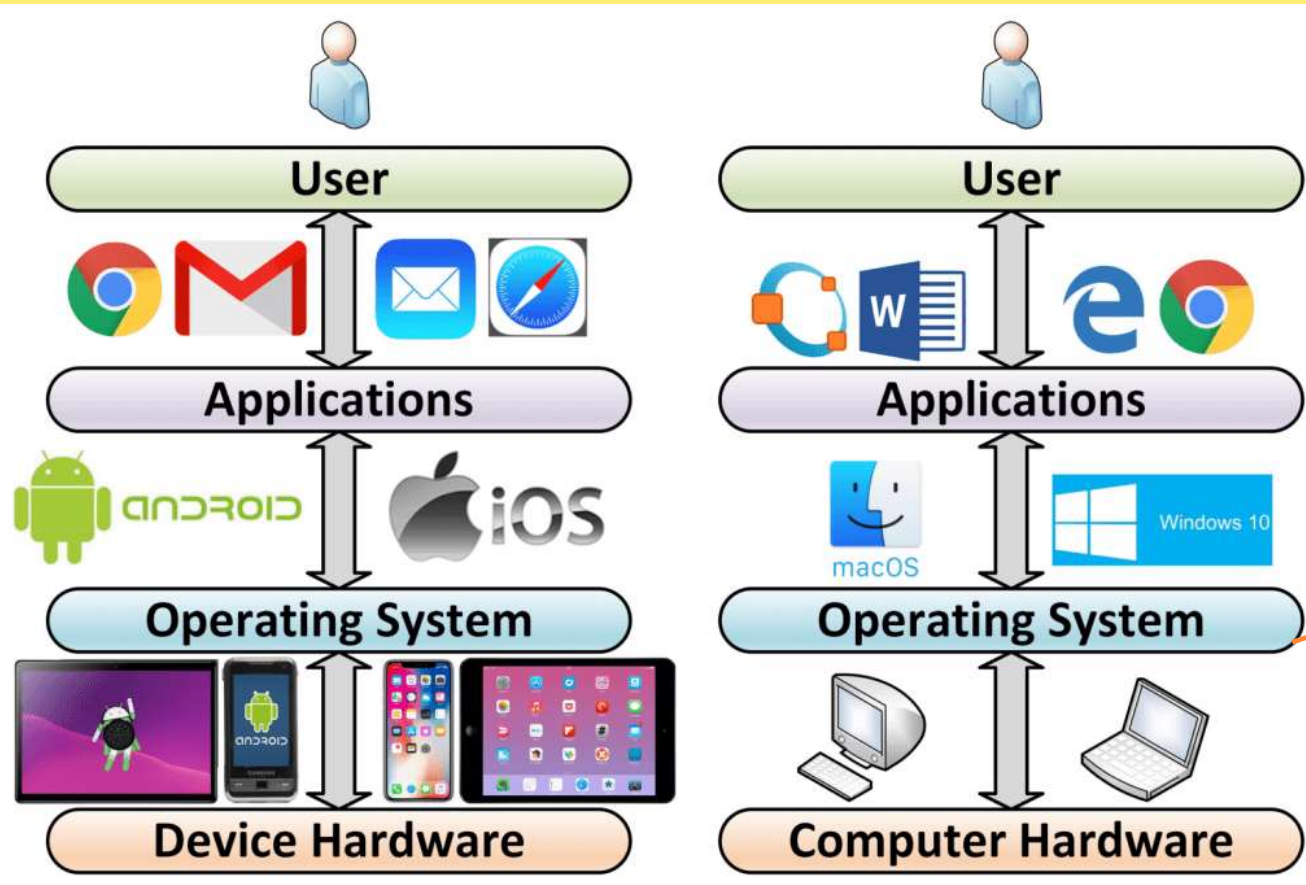


DVD
(DIGITAL VERSATILE DISC)



RAM
(RANDOM ACCESS MEMORY)

BASIC CONCEPTS



BASIC CONCEPTS

❖ Definitions 1:

- ❖ Computer science is the study of **computers** and computational systems. It covers the theory, design, development, and application of the **software** and **hardware** components that make up these systems.
- ❖ Computer science is the study of **computation**, **information**, and **automation**. It spans theoretical disciplines (such as **algorithms**, theory of computation, and information theory) to applied disciplines (including the design and implementation of **hardware** and **software**).
 - *L'informatique est la science du traitement automatique de l'information [Fr].*

BASIC CONCEPTS

- ❖ Automatic information processing involves executing instructions (elementary and complex operations) on input data to generate other information called results or output data. This processing corresponds to an **algorithm**.



- ❖ **Definition 2:** An *algorithm* is an ordered sequence of instructions or actions that outlines the procedure (method) to solve a problem or perform a task.
- ❖ The term algorithm comes from the Latinized name of the Persian mathematician Al-Khawarizmi, known as "the father of algebra".



BASIC CONCEPTS

- ❖ **Definition 2:** *An **algorithm** is an ordered sequence of instructions or actions that outlines the procedure to solve a problem or perform a task.*



- ❖ When writing an algorithm, the following questions should be considered:
 - What is the expected result?
 - What data is required (necessary information)?
 - How to proceed (the treatment to be performed)?

BASIC CONCEPTS

❖ Example 1: A cooking recipe (How to make flan)

matériel nécessaire :



FLAN EN POUDRE



LAIT

étapes

1. mélanger 1 sachet de flan +
lait froid

2. bien mélanger au fouet

3. faire chauffer en remuant jusqu'à épaississement

4. verser dans des ramequins
et laisser refroidir

résultat :



Data (Ingredients)

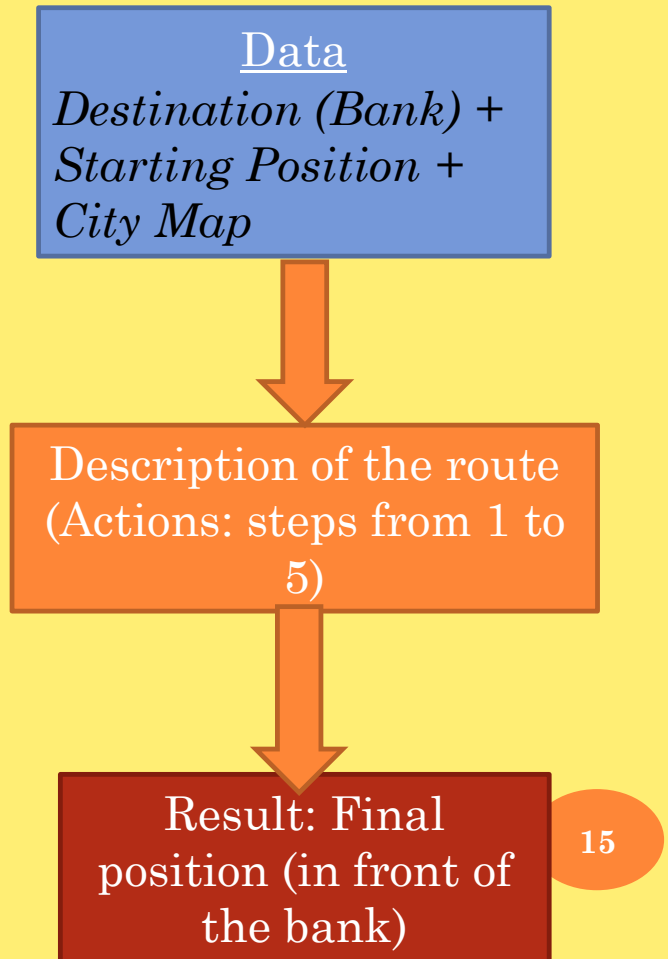
Preparation Methods
(Actions: steps from 1 to
4)

Result (Flan to
eat)

BASIC CONCEPTS

❖ Example 2: Guiding a Visitor

- ❖ Suppose a visitor comes to you and asks for directions to the bank.
- ❖ Since you know its exact location, you might say, for example:
 1. Go straight.
 2. At the second intersection, turn left.
 3. Go straight.
 4. At the first intersection, turn right.
 5. After 50 meters, you'll find the bank on your right.



BASIC CONCEPTS

❖ Example 3: Calculate a student's average for a set of modules.

Module	M1	M2		Mn
Value	X1	X2		Xn
Coefficient	C1	C2		Cn

$$\text{Weighted average} = \frac{\text{coefficients} * \text{values}}{\text{sum of coefficients}}$$

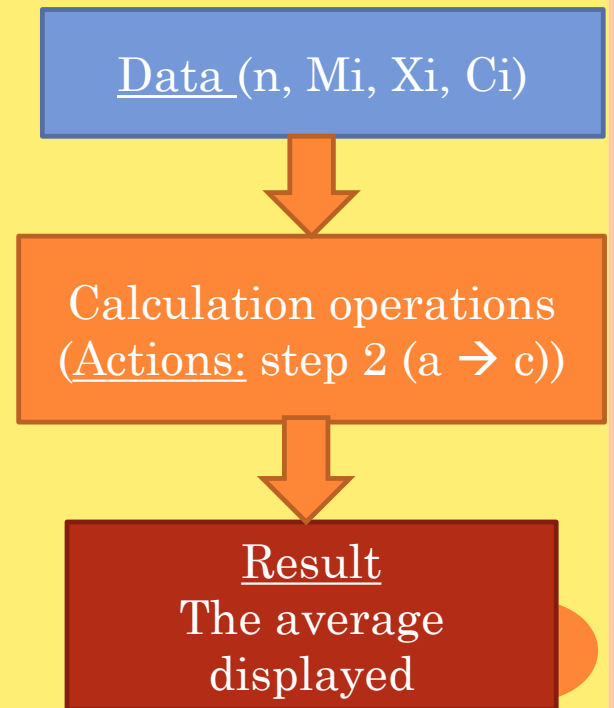
$$\text{Weighted average} = \frac{C1 * X1 + C2 * X2 + \dots + C * Xn}{C1 + C2 + \dots + Cn} = \frac{\sum_{i=1}^n Ci * Xi}{\sum_{i=1}^n Ci}$$

BASIC CONCEPTS

❖ Example 3: Calculate a student's average for a set of modules.

❖ So, we must :

1. Define the number of modules involved (**denoted as n**) as well as the score (**X_i**) and the coefficient (**C_i**) of each module (**M_i**).
2. Perform the following operations:
 - a. Multiply each module's score by its coefficient.
 - b. Calculate the sum of the multiplication results.
 - c. Divide the sum by the total coefficients.
3. Display the student's average (final result).



BASIC CONCEPTS

❖ Example 4: Solving a quadratic equation (second degree)

Standard Form of a Quadratic Equation

$$ax^2 + bx + c = 0$$

Diagram illustrating the components of the standard form of a quadratic equation:

- a is the **Coefficient of x^2** .
- b is the **Coefficient of x** .
- c is the **Constant**.

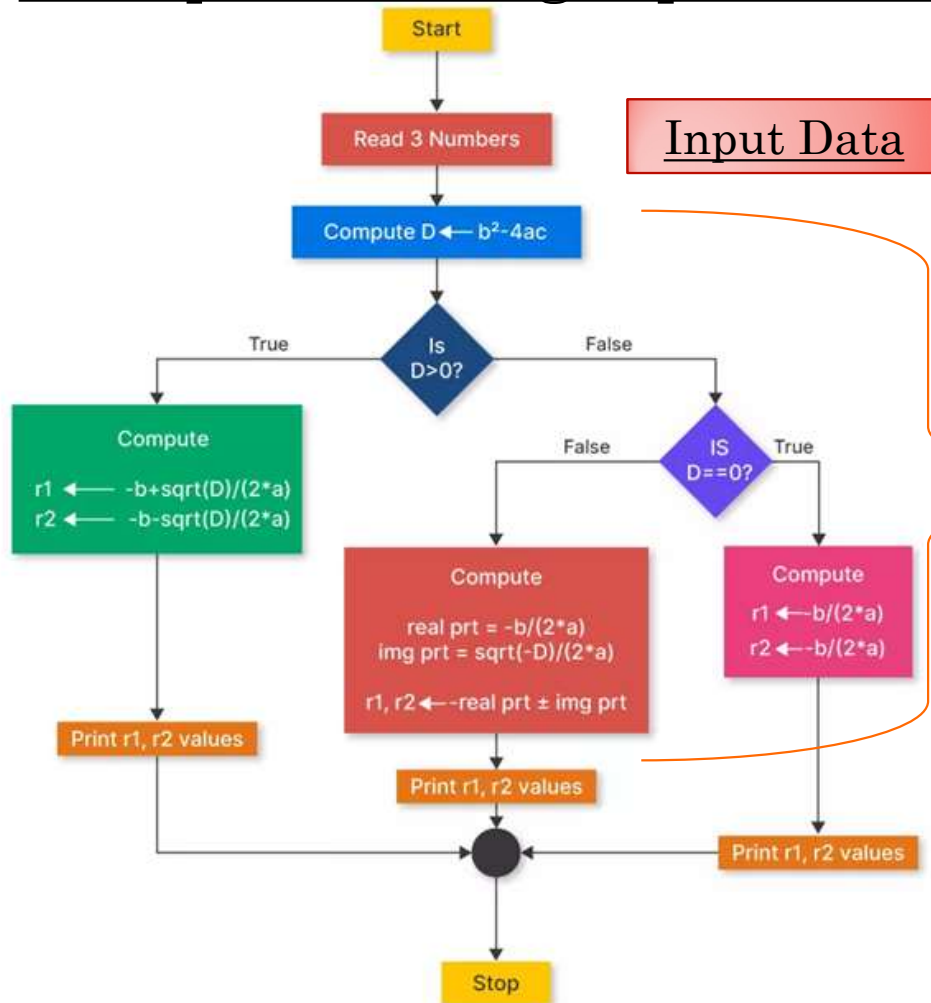
$$0 = ax^2 + bx + c$$

Diagram illustrating the components of the standard form of a quadratic equation:

- 0 is the **Expression set equal to 0**.
- ax^2 is the **Quadratic term $a \neq 0$** .
- bx is the **Linear term**.
- c is the **Constant term**.

BASIC CONCEPTS

❖ Example 4: Solving a quadratic equation (second degree)



Actions:

1. Calculate Δ
2. Depending on Δ , display the solution(s) if it exists.

Results: Display messages and solutions.

BASIC CONCEPTS

- ❖ **Definition 3:** *An algorithm is a sequence of **primitive actions** that, when correctly executed by a **processor**, will yield the desired result.*
 - A **processor** can be any entity capable of understanding and executing the actions constituting an algorithm. It can be a person, an electronic device (alarm), a mechanical device (vending machine), or a computer.
 - The set of objects (elements) necessary to carry out a task described by an algorithm is called the **environment**.
 - An **action** is a step in an algorithm. It is a finite-duration event that modifies the environment.

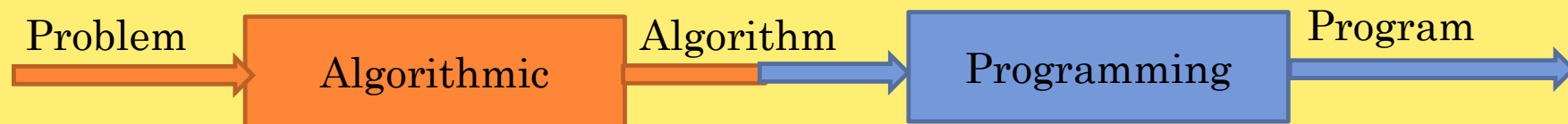
BASIC CONCEPTS

- ❖ **PROPERTIES:** For an algorithm to be valid, it must meet the following criteria:
 - **Correctness:** It must yield the correct result (correct logic).
 - **Exactness:** It must account for all possible cases, handling both the general and specific cases.
 - **Finiteness:** It must terminate after a finite number of actions.
 - **Precision:** It should clearly specify the inputs (data entry) and outputs (result display).
 - **Efficiency:** It should return results within an acceptable time, regardless of the size of the input data.
 - **Independence:** It should be independent of programming languages and computer hardware.

BASIC CONCEPTS

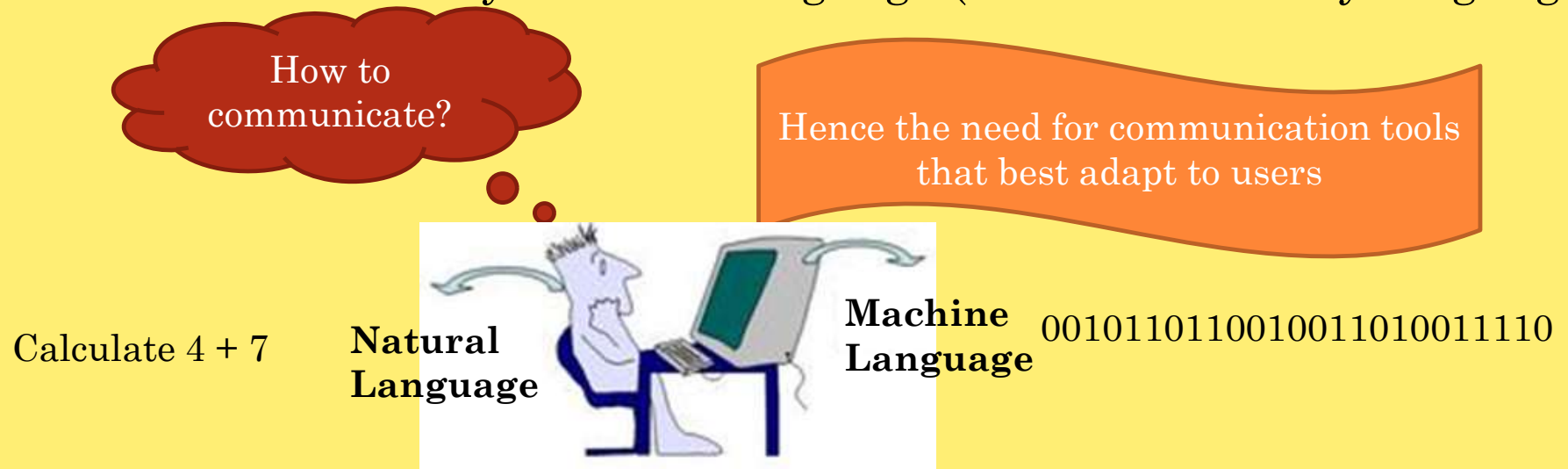
❖ To solve a given problem, two phases are necessary:

1. Algorithmic phase: Converts a problem to be solved into an algorithm that describes the problem-solving process.
 - **Definition 4:** *Algorithmic is the science of algorithms, focusing on the art of constructing algorithms and determining their validity, robustness, reusability, complexity, and efficiency.*
2. Programming phase: Translates an algorithm into a language "understandable" by the computer for automatic execution.



BASIC CONCEPTS

- ❖ **Definition 5:** A **computer** is a machine capable of automatically performing arithmetic and logical operations based on programs that define the sequence of these operations.
- ❖ A computer understands only machine language (also called binary language).

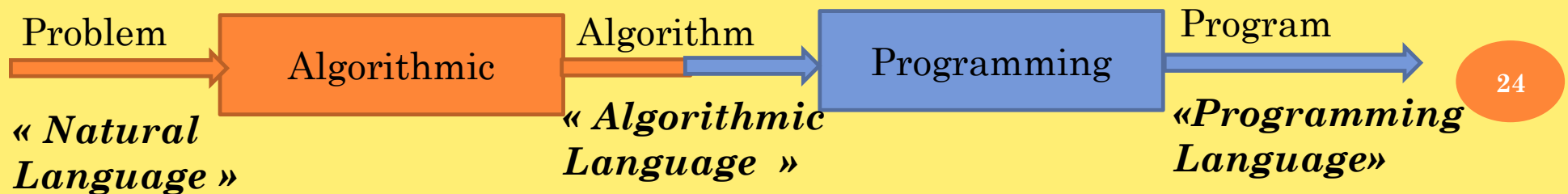


- Programming languages allow the translation of algorithmic language into a binary program for the computer.

PROGRAMMATION

❖ Definitions 6:

- ❖ **An Algorithmic language** is a set of rules for correctly writing an algorithm.
- ❖ **Programming** is the translation of the algorithm into a programming language.
- ❖ **A Program** is a sequence of instructions written in a programming language that translates an algorithm. Each instruction specifies the operation the computer must perform.



PROGRAMMING

❖ Definitions 6:

- ❖ A **programming language** is a set of rules and conventions used to write instructions that will be executed by a computer. It is a means of communication between programmers and machines, and allows developers to create software, applications, websites, and other types of programs. es applications, des sites web et d'autres types de programmes.

Level	Examples	Characteristics
Machine Language	Binary (0 and 1)	Understood only by the processor, unreadable by humans.
Low-Level Language	Assembly Language (or Assembler)	Close to the hardware, one assembly instruction = one machine instruction.
Intermediate-Level Language (or Middle-Level Language)	C	Balance: readable by humans but allows direct control over memory.
High-Level Language	Python, Java, C#, Ruby...	Abstract, easy to write, but require complete translation into binary.

PROGRAMMATION

- **Low-level language**

For humans: difficult and time-consuming to write (assembly, machine code).
For the computer: very fast execution.

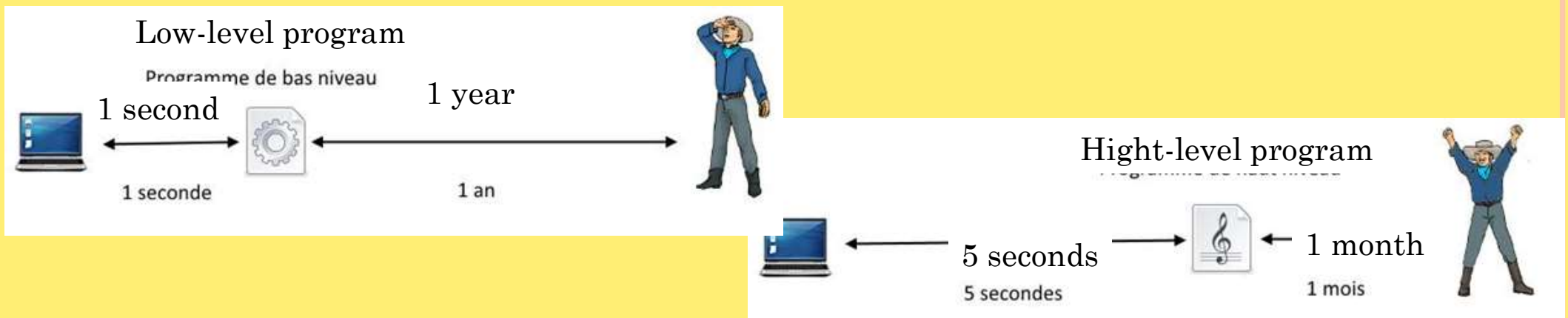
Conclusion: fast for the machine, but complicated for the programmer.

- **High-level language**

For humans: easy and quick to write (C, Python, Java...).

For the computer: requires translation into binary → a bit slower.

Conclusion: fast for the programmer, but requires translation for the machine.



PROGRAMMATION

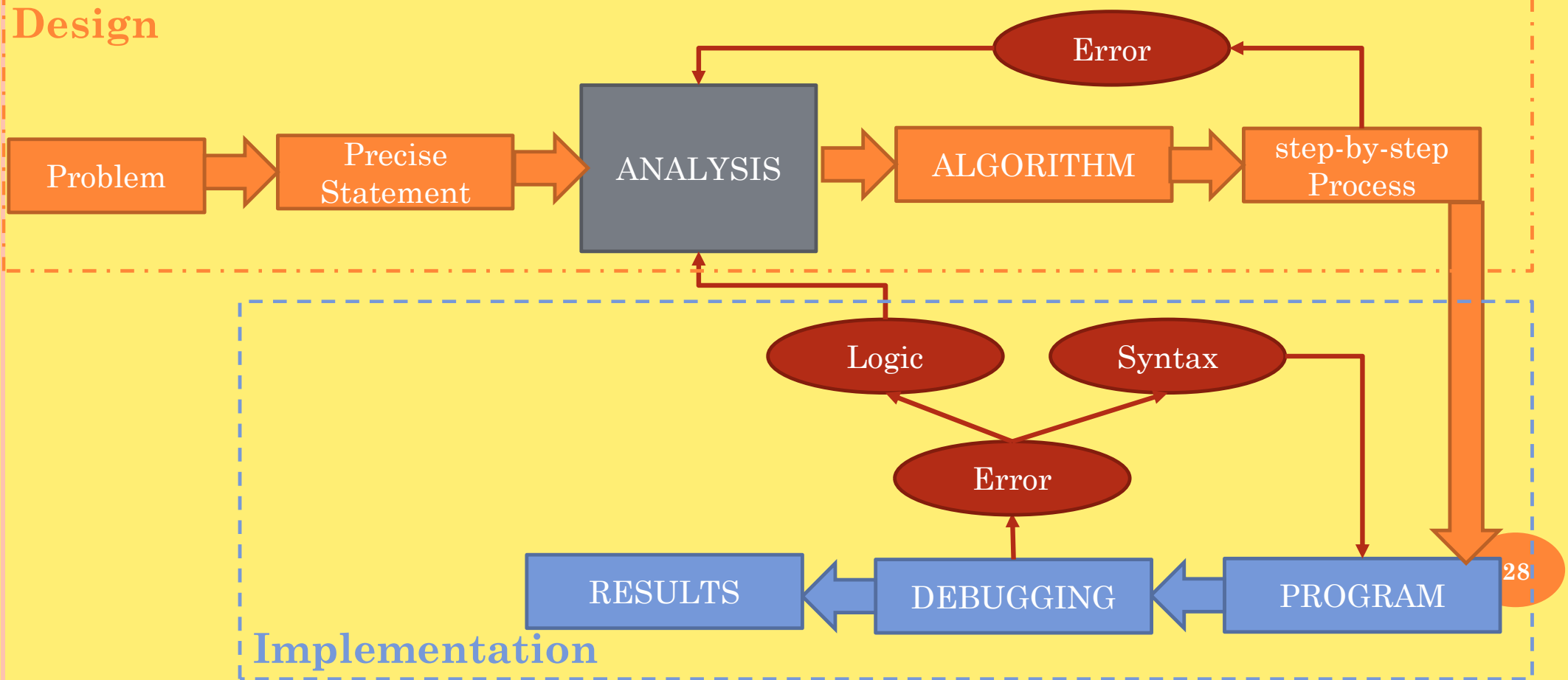
❖ The most used programming languages in May 2025

Rank	Language	Part (%)
1	Python	25,87 %
2	C++	10,68 %
3	C	9,47 %
4	Java	8,84 %
5	C#	4,69 %
6	JavaScript	3,21 %
7	Go	2,28 %

PROBLEM SOLVING STEPS

❖ Solving steps

Design



PROBLEM SOLVING

- ❖ The **analysis** MUST be carried out BEFORE the algorithm is built. This involves :
 - ✓ giving the basic idea(s),
 - ✓ formulating them in simple, concise, clear sentences or with a diagram
 - ✓ And then structuring them.
 - Don't hesitate to use your own conventions (drawings, diagrams, narrations, colours,).
- ❖ Important tip: use a well-chosen example to explain your idea.
- ❖ Validate the analysis by asking :
 1. Is the basic idea correctly and simply expressed?
 2. Are the derived ideas well-structured?
 3. Have I avoided unnecessary details that might confuse the analysis?

PROBLEM SOLVING

- ❖ The step by step process consists of running the algorithm, action by action, by hand, to check that it gives the desired result.
- ❖ It can be summarised in a table in which each column represents an object in the environment and each row an action in the algorithm.

Actions \ Objects	Objects	
	Object 1	
1		
2		
.....		

PROBLEM SOLVING

❖ Example 5 (solution 1):

- ❖ Problem: Find the divisors of a number.
- ❖ Precise Statement: Given a non-zero natural number N , give (or construct) the algorithm that provides a list of its divisors.
- ❖ Analysis :
 - Let N be a non-zero natural number.
 - Divide N successively by $i = 1, 2, 3, \dots, N$.
 - Whenever the remainder of the division of N by i is 0 (i.e., i is a divisor),
 - Print i

PROBLEM SOLVING

- ❖ **Example 5 (solution 1):**
- ❖ **Problem:** Find the divisors of a number.
- ❖ **Precise Statement:** Given a non-zero natural number N , give (or construct) the algorithm that provides a list of its divisors.
- ❖ **Analysis 1 (more detailed):**
- ❖ Let N be a non-zero natural number.
- ❖ Generate divisors i from 1 to N :
 - To generate the divisors i , add 1 each time to i , which is 1 at the start.
- ❖ For each divisor generated:
 - divide N by i .
 - Keep the remainder of the division.
 - If it is 0, then i is a divisor and must be printed.

PROBLEM SOLVING

❖ Example 5 (solution 2):

- ❖ Knowing that the number 1 is a divisor of any number and that a number N is a divisor of itself [*special cases*], we can vary the loop i from 2 to $N/2$ to find the other divisors [*general case*].
 - Let N be a non-zero natural number.
 - **Print (1)**
 - **Generate divisors i from 2 to $(N \text{ div } 2)$**
 - For each divisor generated
 - Divide N by i .
 - Keep the remainder of the division
 - If it is equal to 0, then i is a divisor and must be printed.
 - **Print (N)**

PROBLEM SOLVING

❖ Example 5:

❖ Analysis 1:

- ❖ Let N be a non-zero natural number.
- ❖ Generate divisors i from 1 to N .
- ❖ For each divisor generated:
 - divide N by i .
 - If the remainder of the division is 0, then i is a divisor and must be printed.

Algorithm Diviseurs

Var

N, i : Integer

Begin

scan (N)

For $i \leftarrow 1$ **to** N **do**

If $((N \bmod i) = 0)$ **then print** (i)

End.

PROBLEM SOLVING

❖ Example 5: Step-by-step Process (N = 21):

N	i	Remainder N/i	Print (i)
21			
	1	0	1
	2	1	
	3	0	3
	4	1	
	5	1	
	6	3	
	7	0	7
	8	5	
	9	3	
	10	1	
	11	10	

N	i	Remainder N/i	Print (i)
	12	9	
	13	8	
	14	7	
	15	6	
	16	5	
	17	4	
	18	3	
	19	2	
	20	1	
	21	0	21

PROBLEM SOLVING

❖ Example 5:

Algorithm Diviseurs

Var

N, i : Integer

Begin

scan (N)

For i ← 1 to N **do**

If ((N mod i) = 0) **then print** (i)

End.



```
1  #include <stdio.h>
2
3  int main()
4  {
5      int N, i;
6      printf("donner la valeur de N = ");
7      scanf("%d", &N);
8      printf("La liste des diviseurs de N : ");
9      for (i = 1; i <= N; i++)
10         if (N % i == 0)
11             printf("%d , ", i);
12     return 0;
13 }
```

C Program



```
donner la valeur de N = 21
La liste des diviseurs de N : 1 , 3 , 7 , 21 ,
Process returned 0 (0x0)    execution time : 2.517 s
Press any key to continue.
```

Result of
execution (in
console)

COURSE SOURCES

- S.Aroussi (2024) , Algorithms and data structure, 1st-year engineer, Computer Science department at Blida University.
- B. CHERGOU (2014) & N. MEDJAOUI (2018), *Cours d'algorithmique/pascal*, 1st-year CPI, Higher National School of Computer Science (ESI) Oued Smar, Algiers.
- M. ZAIR (2022), *Algorithmique 1*, 1st-year License Common Core MI (Mathematics and Computer Science), Faculty of Sciences, Saad Dahlab University, Blida.
- O. Hadj Yahia (2022), *Bureautique, algorithmique et langage de programmation*, 1st-year License Common Core SM, Faculty of Sciences, Saad Dahlab University, Blida.



Saad Dahlab University - Blida1
Faculty of Sciences
Department of Computer Science
Computer Science Engineering
Semester 1 (1st year)
Algorithmic and Data Structures 1



CHAPTER II: SIMPLE SEQUENTIAL ALGORITHM

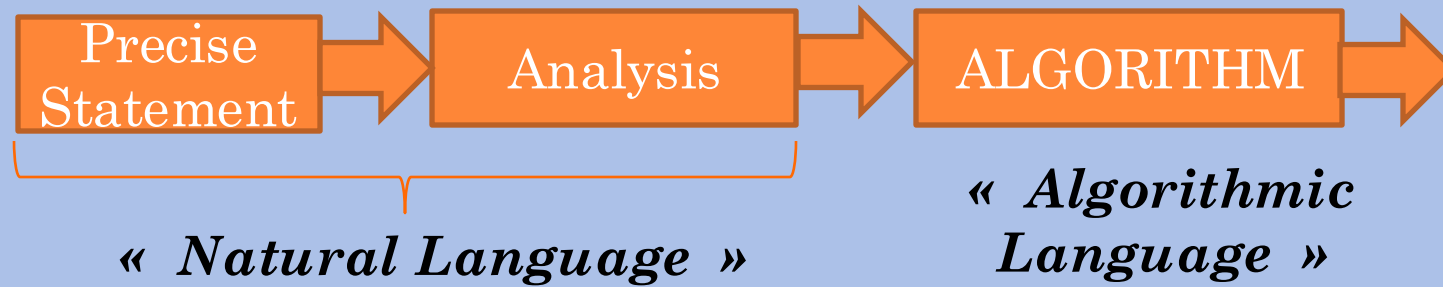
Dr. MUSTAPHA CHABA MOUNA 2025-2026

CONTENT

- ❑ Need for an algorithmic formalism.
- ❑ General structure of an algorithm.
- ❑ Objects in the algorithmic environment: internal objects, constants, variable declarations, identifiers, simple types.
- ❑ Body of the algorithm: elementary actions (Read, Write, Assignment, Expressions) and control structures (alternative and iterative).
- ❑ Flowchart

NEED FOR AN ALGORITHMIC FORMALISM

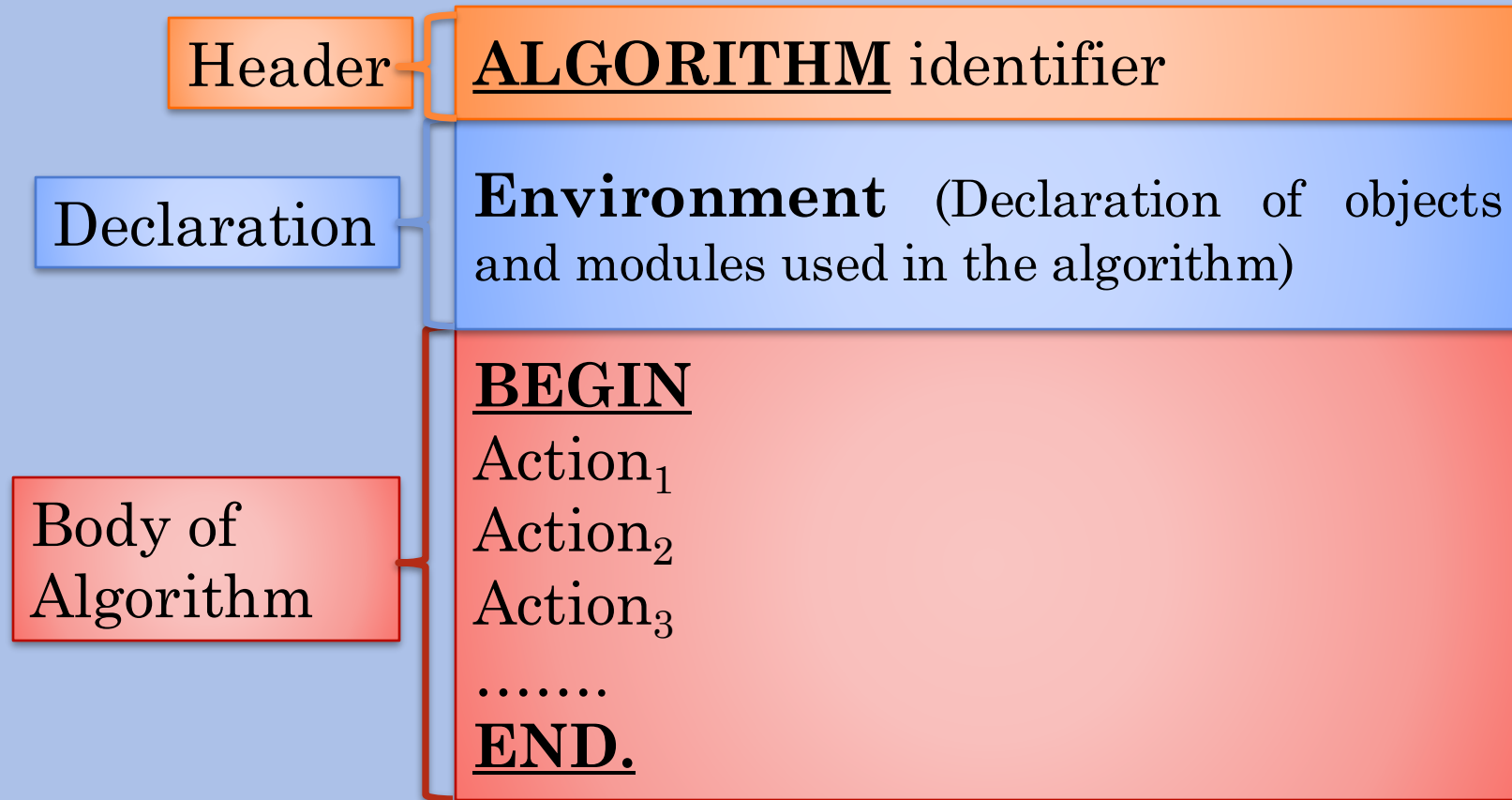
- ❖ Once a problem has been analyzed, the designer will need to express their solution in a **universal formalism** in the form of an **algorithm**.



- ❖ The problem is to use a common language to understand algorithms built by others and vice versa. Hence the importance of formalism.
- ❖ **Definition.** An algorithmic formalism is a set of conventions (or rules) in which any solution to a given problem is expressed. It is a common and precise language that allows for unambiguous communication.

GENERAL STRUCTURE OF AN ALGORITHM

- ❖ An algorithm is organized (structured) into three parts:



Reserved words (keywords): ALGORITHM, BEGIN, END,

GENERAL STRUCTURE OF AN ALGORITHM

- ❖ An algorithm is organized (structured) into three parts:
 1. Header: Defined by the reserved word **ALGORITHM**, followed by an identifier representing the **name** of the algorithm; it has other functionalities with the modules.
 - A reserved word is a language-specific word used to define parts of the algorithm, to use or construct types, and to order operations or actions,
 2. Environment: Includes all the objects (elements) needed to execute the algorithm. Each object must be declared with its identifier and type.
 3. Body of algorithm: Composed of several elementary or composite actions (or instructions) arranged chronologically, indicating the procedure to achieve the desired (expected) result.

ENVIRONMENT OBJECTS

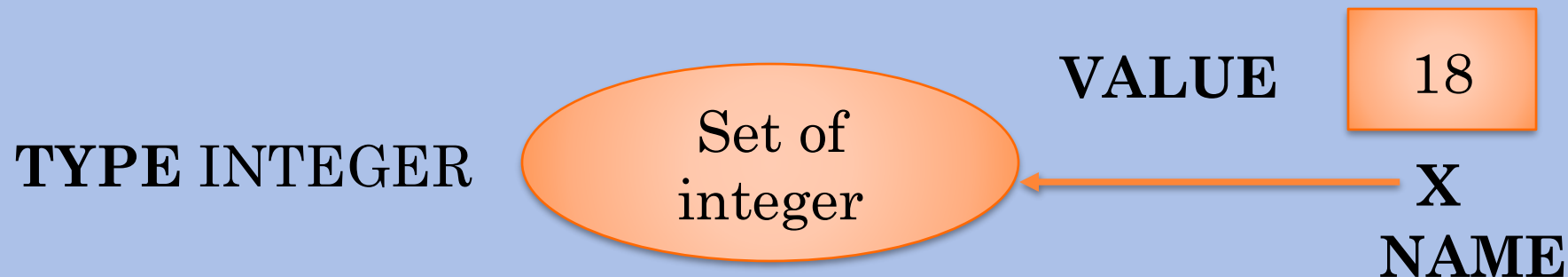
- ❖ For each algorithm, the following objects are distinguished:



1. Input objects: Provided to the algorithm.
2. Output objects: The results produced by the algorithm.
3. Internal objects, which are the algorithm's operating objects, used for internal manipulations.
 - The set of input and output objects constitutes the **parameters** of the algorithm (*which is an important concept in module construction*).
 - In the following, we focus on internal objects: variables and constants.

OBJECTS DECLARATION

- ❖ Every algorithm uses objects, which are declared in its environment.
- ❖ Each object must be matched with:
 - A NAME (also called an IDENTIFIER), used to designate and distinguish it from other elements.
 - A TYPE, indicating the nature of the set in which the object takes its values.
 - A VALUE assigned to the object at a given moment.



OBJECTS DECLARATION (IDENTIFIERS)

- ❖ There are some constraints and conventions to respect for naming objects:
 - Only letters (lowercase and uppercase) and numbers (abcABC012) are allowed.
 - The name must start with a letter.
 - Spaces are not allowed; instead, the underscore "_" can be used (this is the only special character allowed).
 - Accents (e.g., é, à, ê) are not allowed.
 - The name must be different from the reserved words (keywords) of the algorithmic or programming language. Examples of reserved words in algorithmic language: **algorithm, begin, end, if, else, then, while, for, do, to, function, var, const, type, array, real, integer, boolean, char, string, true, false, and, or, not...**

OBJECTS DECLARATION (IDENTIFIERS)

❖ **Rule to remember:** *An identifier starts with an alphabetic letter, followed or not by other letters, numbers, or underscores.*

❖ **Example:** Consider the following list of words:

noteInfo mail nom bac2014 numero_de_telephone
 Prix-TTC prénom x1 moy Moy_1 #B
 A.X 1B2 x 1 Xx25ab 2y -0.5 1,8 fab & '5'
"vap" 'd" 0 'M' alpha 25,3 14.5 -11 c/n #ff a*b
false y² g! Ajou5 +9 +10.6 "k=14.9" "15" true
Algorithm gt-at Fi xy<5 dy cinq google.fr X_1 a₁ "-
11,6" 'vap' vap '#' "R "

1. Which of these are correct identifiers?

OBJECTS DECLARATION (IDENTIFIERS)

❖ **Rule to remember:** *An identifier starts with an alphabetic letter, followed or not by other letters, numbers, or underscores.*

❖ **Example:** Consider the following list of words:

1. Correct object identifiers (names):

noteInfo mail nom bac2014 numero_de_telephone
Prix-TTC prénom x1 moy Moy_1 #B
A.X 1B2 x 1 Xx25ab 2y -0.5 1,8 fab & '5'
"vap" 'd" 0 'M' alpha 25,3 14.5 -11 c/n #ff a*b
false y² g! Ajou5 +9 +10.6 "k=14.9" "15" true
Algorithm gt-at Fi xy<5 dy cinq google.fr X_1 a₁ "-
11,6" 'vap' vap '#' "R"

OBJECTS DECLARATION

❖ The declaration of objects follows this order:

- Constants declarations *
- Types declarations *
- Variables declarations *
- Subprograms (Modules) declarations
- Labels declarations

❖ In what follows, we will only address the declarations marked with an (*).

ALGORITHM identifier

Environment

Constants declarations (**CONST**)

Types declarations (**TYPE**)

Variables declarations (**VAR**)

Modules declarations

Labels declarations

BEGIN

Action₁

Action₂

Action₃

.....

END.

CONSTANTS DECLARATION

- ❖ A **constant** is an object whose value cannot be changed during the execution of the algorithm.

CONST Identifier_constant = value

Example:

```
CONST Pi =3.14  
CONST title = " Results"  
CONST hundred = 100  
CONST comma =','
```



Example:

```
CONST  
Pi =3.14  
title = " Results"  
hundred = 100  
comma =','
```

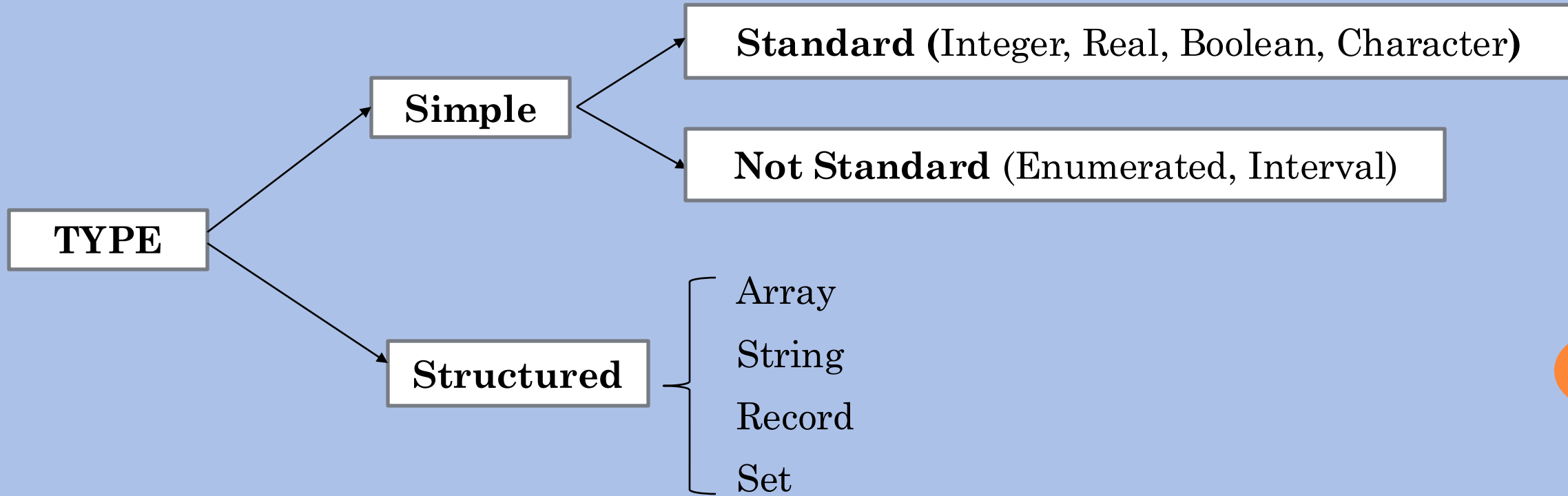
- Note: if there are several constants to declare, the word “**CONST**” can only be written once.

VARIABLES DECLARATION

- ❖ A **variable** is an object whose value can be changed at any time.

VAR Identifier_variable : **TYPE**

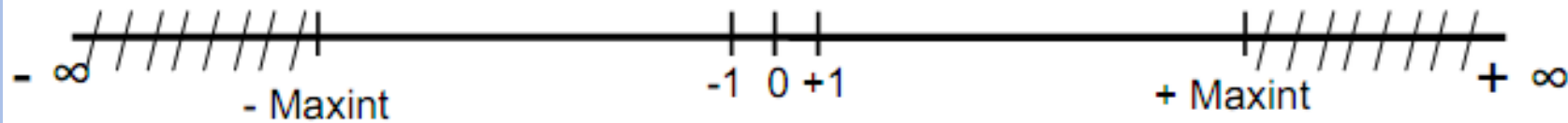
- The **TYPE** defines the set of values an object can take.



SIMPLE TYPES

- ❖ The **integer** type represents integer numerical values that can be positive, negative or zero.

Value domaine :



Variable Type	Keyword	Bytes Required	Range
Character	char	1	-128 to 127
Unsigned character	unsigned char	1	0 to 255
Integer	int	2	-32768 to 32767
Short Integer	short int	2	-32768 to 32767
Long Integer	long int	4	-2,147,483,648 to 2,147,438,647
Unsigned Integer	unsigned int	2	0 to 65535
Unsigned Short integer	unsigned short int	2	0 to 65535
Unsigned Long Integer	unsigned long int	4	0 to 4,294,967,295

SIMPLE TYPES

- ❖ The **real** type represents floating-point numbers: real numbers with a fractional (decimal) part.

(+/-) integer_part . Decimal_part

- ❖ Real types represent floating-point numbers:

Type	Interval	Length (bytes)
Real	$1.5 * 10^{-45} .. 3.4 * 10^{38}$	6
Single	$2.9 * 10^{-39} .. 1.7 * 10^{38}$	4
Double	$5.0 * 10^{-324} .. 1.7 * 10^{308}$	8

- ❖ The **Boolean** type represents two logical values: **TRUE** or **FALSE**.

SIMPLE TYPES

- ❖ The **character** type (**CHAR**) includes all characters from the system's character set: alphabetic and numeric characters, special symbols, and the white space (or space) character.
- ❖ A character is represented by the character itself enclosed in single quotes (apostrophes) ' '. For example: 'a', '9', '#'.
- ❖ The values are ordered according to the internal character codes, such as ASCII (American Standard Code for Information Interchange)

32	48 0	64 @	80 P	96 `	112 p
33 !	49 1	65 A	81 Q	97 a	113 q
34 "	50 2	66 B	82 R	98 b	114 r
35 #	51 3	67 C	83 S	99 c	115 s
36 \$	52 4	68 D	84 T	100 d	116 t
37 %	53 5	69 E	85 U	101 e	117 u
38 &	54 6	70 F	86 V	102 f	118 v
39 '	55 7	71 G	87 W	103 g	119 w
40 (	56 8	72 H	88 X	104 h	120 x
41)	57 9	73 I	89 Y	105 i	121 y
42 *	58 :	74 J	90 Z	106 j	122 z
43 +	59 ;	75 K	91 [	107 k	123 {
44 ,	60 <	76 L	92 \	108 l	124
45 -	61 =	77 M	93]	109 m	125 }
46 .	62 >	78 N	94 ^	110 n	126 ~
47 /	63 ?	79 O	95 _	111 o	127 "

OBJECTS DECLARATION (IDENTIFIER)

❖ **Example :** Consider the following list of words:

noteInfo mail nom bac2014 numero_de_telephone
 Prix-TTC prénom x1 moy Moy_1 #B
 A.X 1B2 x 1 Xx25ab 2y -0.5 1,8 fab & '5'
 "vap" 'd" 0 'M' alpha 25,3 14.5 -11 c/n #ff a*b
 false y² g! Ajou5 +9 +10.6 "k=14.9" "15" true
 Algorithm gt-at Fi xy<5 dy cinq google.fr X_1 a₁ "-
 11,6" 'vap' vap '#' "R "

1. Which of these are **correct identifiers**?
2. Complete the table by specifying the type of the correct values of objects?

Integer	Real	Boolean	Character

OBJECTS DECLARATION (IDENTIFIER)

❖ **Example :** Consider the following list of words:

noteInfo mail nom bac2014 numero_de_telephone
 Prix-TTC prénom x1 moy Moy_1 #B
 A.X 1B2 x 1 Xx25ab 2y -0.5 1,8 fab & '5'
 "vap" 'd" 0 'M' alpha 25,3 14.5 -11 c/n #ff a*b
 false y² g! Ajou5 +9 +10.6 "k=14.9" "15" true
 Algorithm gt-at Fi xy<5 dy cinq google.fr X_1 a₁ "-
 11,6" 'vap' vap '#' "R "

1. Which of these are **correct identifiers**?
2. Complete the table by specifying the type of the correct values of objects?

Integer	Real	Boolean	Character
0 ; -11; +9	-0.5 ; 0 ; 14.5 ; 11; +9 ; +10.6	false ; true	'5' ; 'M' ; '#'

SIMPLE TYPES

- ❖ Sometimes we define types that are specific to our problem using **not standard** types.

TYPE

```
Identifier_Type = Type_not_standard
```

- Standard types (like integer, real, boolean, and character) do not require explicit declaration in most programming languages.
- ❖ There are two not standard types:
 - **Enumerated** type: Defines an ordered set of values designated by identifiers (up to 256 constants).
 - **Interval** type: Defines an interval within a set of ordered values or predefined by an ordinal type with lower and upper bounds.

SIMPLE TYPES

- ❖ Enumerated type defines an ordered set of values designated by identifiers (up to 256 constants).

- ❖ **Example :**

TYPE

days = (Sun, Mon, Tue, Wed, Thu, Fri, Sat)

colour = (violet, orange, blue, green, yellow, red)

size = (large, medium, small)

answer = (yes, no)

holiday = (aïd , mouloud , achoura , moharram)

month = (sept, oct, nov, dec, jan, feb, mar, apr, may, jun, jul, aug)

SIMPLE TYPES

- ❖ **Interval** type defines an interval within a set of ordered values or predefined by an ordinal type with lower and upper bounds.

- ❖ **Example :**

TYPE

index = 1 .. 10

digit = '0' .. '9'

lettre_upper = 'A' .. 'Z'

lettre_lower = 'a' .. 'z'

- ❖ It is possible to deduce a not standard type from another type.

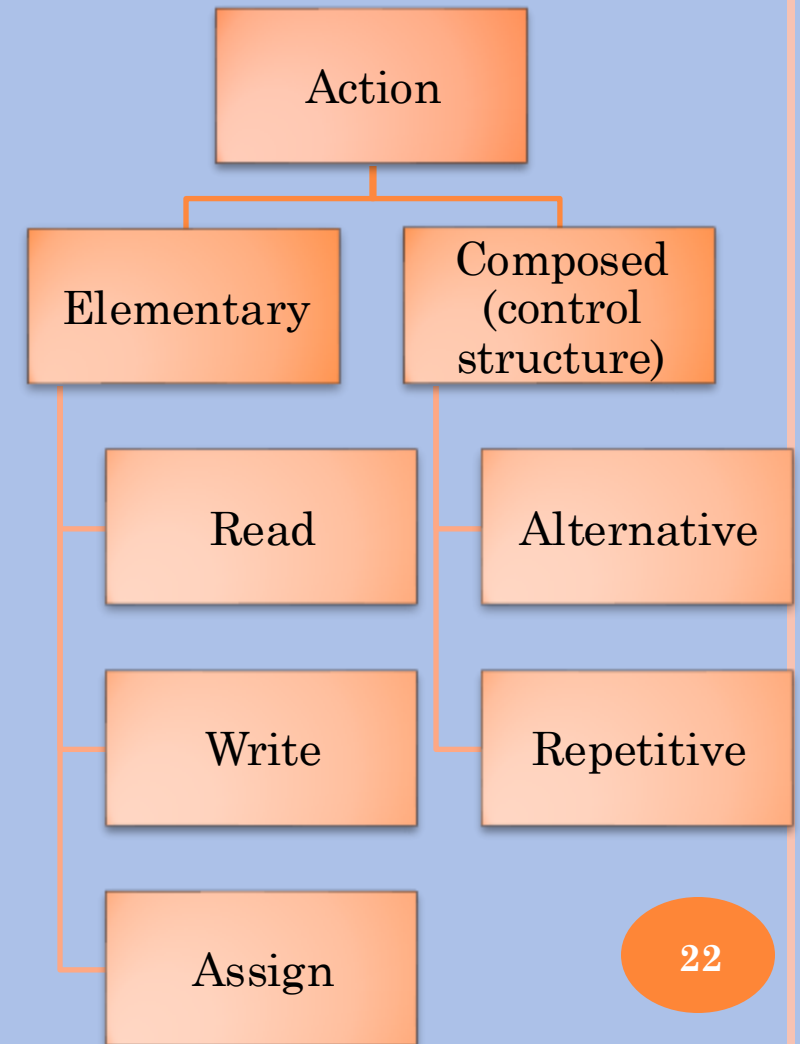
TYPE

month = (sept, oct, nov, dec, jan, feb, mar, apr, may, jun, jul, aug) //enumeration

Annee_univ = sept..jun //interval

ALGORITHM BODY

- ❖ The algorithm's body contains a sequence of actions (instructions) executed in the order they appear in the algorithm.
- ❖ An **action** is an event that operates on the objects of the algorithm, modifying their values or observing them.
- ❖ There are two types of action:
 - ❖ Elementary action (also called primitive) that a processor can execute without any additional information.
 - ❖ An action composed of several other basic actions which is executed under certain conditions one or more times. These are known as control structures.



ELEMENTARY ACTIONS (READING)

- ❖ Reading allows to read a value from an input device (usually the keyboard), and then store the read value in a variable. If you have several variables to read, just separate them with ','.

SCAN ([f], Var₁ , Var₂ , ... , Var_n)

- f indicates the logical name of the input file. By default, this is the keyboard.
 - Reading blocks execution until values are received
 - The values which are read from the keyboard are assigned respectively to the variables, taking into account the compatibility of the types
- ❖ Example: scan (N) ; scan (a,b,c)

ELEMENTARY ACTIONS (WRITING)

- ❖ Writing displays the values of the objects mentioned in the list. Objects are separated by ','.

PRINT ([f], R_1 , R_2 , ... , R_n)

- f indicates the logical name of the input file. By default, this is the screen.
- ❖ An object can be:
 - A value: like 5, “hello”,.... A value is displayed as it is.
 - A variable: like x, y....a variable is replaced by its value
 - An expression: like $(x+y)/2$, A or B,... The expression is evaluated and the result is displayed.

ELEMENTARY ACTIONS (WRITING)

❖ **Examples:** Give display result of each line

- i. $a \leftarrow 5$; print (a) ;
- ii. $c \leftarrow 4$; print ("c=",c) ;
- iii. $b \leftarrow a*a-c$; print (a*a-c) ;
- iv. print (a, b, c) ;
- v. print (a, b, c) ;
- vi. print (a, ' ', b, ' ', c) ;
- vii. print ("a=", a, " b=", b, " c=", c) ;

Screen :

- i. 5
- ii. c = 4
- iii. 21
- iv. 5214
- v. 5214
- vi. 5 21 4
- vii. a = 5 b = 21 c = 4

ELEMENTARY ACTIONS (WRITING)

- ❖ **Note 1:** It is important to precede any display of a value of a certain variable with a message so that the user understands the displayed values.

➤ **Example 1:** `print (a, b, c)` `print ("a= ", a, "b= ", b, "c = ", c)`

- ❖ **Note 2:** It is important to precede a read instruction with an instruction to display a message to indicate to the user the nature and type of the value to be entered.

➤ **Example 2:** `scan (x, y)`  `print ("give the value of x: ")`
`scan (x)`
`print ("give the value of y: ")`
`scan (y)`

ELEMENTARY ACTIONS (ASSIGNMENT)

- ❖ Assignment (symbolized by the sign ' $\leftarrow$ ') consists of assigning (giving) a value to an object. The value can be a constant, the value of another object, or the result of evaluating an expression.
 - identifier $\leftarrow$ value. **Example:** $x \leftarrow 0$ (set x to zero)
 - identifier $\leftarrow$ identifier. **Example :** $x \leftarrow y$ (put the value of object y in x ; it reads *x receives y*)
 - Note: it is possible to assign an integer value to a real variable, but not vice versa.
 - identifier $\leftarrow$ expression. **Example:**
 - $x \leftarrow x + 1$ (add 1 to x)
 - $x \leftarrow x - y + z$ put in x the result of the evaluation of the expression $x - y + z$

ELEMENTARY ACTIONS (ASSIGNMENT)

- ❖ In the assignment, it will therefore be necessary to evaluate, possibly, the entity to the right of the assignment symbol and then put this result in the entity to the left of the assignment symbol.
 - the type of the expression must be compatible with the type of the left variable.
 - the previous content of the left variable is lost! (overwritten)
- ❖ **Example:** Unroll (step-by-step Process) this alg

```
y ← 2 ;  
x ← y ;  
x ← x*2 ;  
x1 ← y + x ;  
y1 ← x1+2 ;  
y1 ← y1*2 ;  
x ← y1+3 ;
```

Instruction	y	x	x1	y1
y ← 2 ;	2	?	?	?
x ← y ;	2	2	?	?
x ← x*2 ;	2	4	?	?
x1 ← y+x ;	2	4	6	?
y1 ← x1+2 ;	2	4	6	8
y1 ← y1*2 ;	2	4	6	16
x ← y1+3 ;	2	19	6	16

ELEMENTARY ACTIONS (EXPRESSIONS)

- ❖ An expression is a coherent (possibly parenthesized) set of operands and operators that are evaluated to give a value.



- ❖ Operands can be variables, constants, and functions.
- ❖ Operators can
 - arithmetic: - unary (negative digit sign), * (asterisk for product), / (slash for division), div (integer division), MOD (modulo), + (addition), - (subtraction).
 - relational: > (higher), >= (higher or equal), < (lower), <= (lower or equal), = (equal), ≠ (different).
 - logic: NOT (not logical), AND (and logical), OR (or logical).

ELEMENTARY ACTIONS (EXPRESSIONS)

❖ Operators can

- arithmetic: - unary (negative digit sign), * (asterisk for product), / (slash for division), div (integer division), MOD (modulo), + (addition), - (subtraction).
- relational: > (higher), >= (higher or equal), < (lower), <= (lower or equal), = (equal), ≠ (different).
- logic: NOT (not logical), AND (and logical), OR (or logical).
- All operators are binary except two which are unary: - unary and NOT.



- Depending on the type of operator, a distinction is made between arithmetic, relational, logical and mixed expressions.

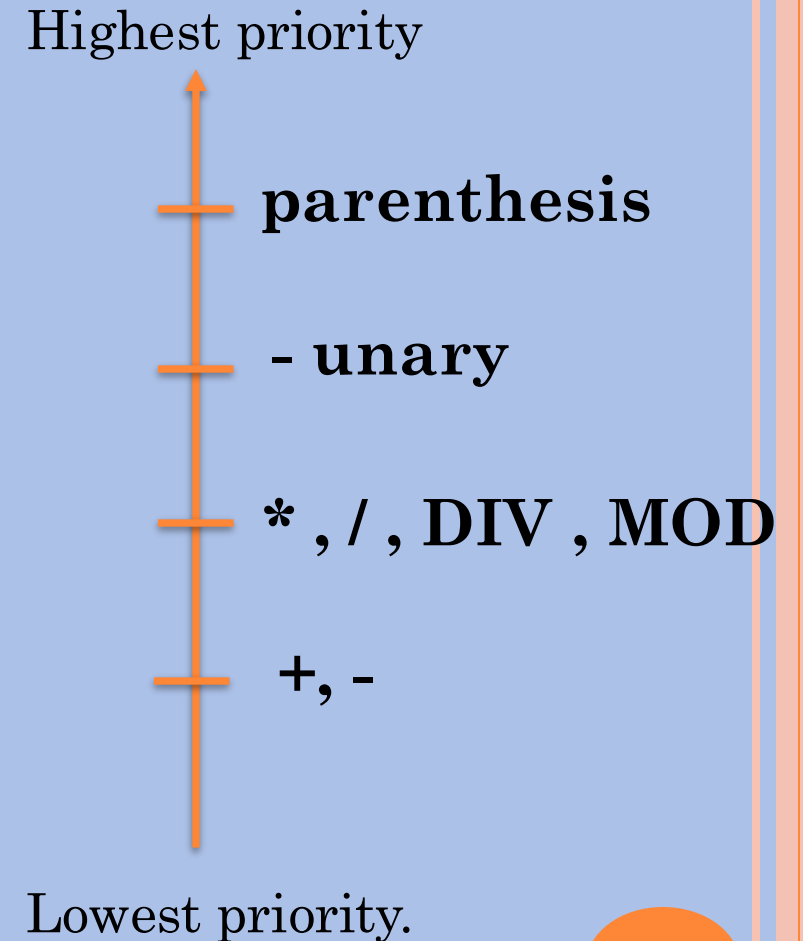
ELEMENTARY ACTIONS (EXPRESSIONS)

- ❖ **Arithmetic** expressions are a finite sequence of arithmetic operators relating to variables or numerical values and which have a numerical value (Integer "I" or Real "R").

Operator and meaning	Types of operands	Type of result	Examples
- unary (sign)	op I op R	I R	-3 -5.9
/ (actual division)	I op I ; I op R ; R op I ; R op R	R	$5/2 = 2.5$
* (product); + (addition); - (subtraction)	I op I E op I ; R op I ; R op R	I R	$5*3 = 15$ $5*3.5 = 17.5$
DIV (integer division quotient)	I op I	I	dividend remainder divisor quotient MOD → 4 34 5 6 ← DIV
MOD (the modulo: remainder of the integer division)			

ELEMENTARY ACTIONS (EXPRESSIONS)

- ❖ The hierarchy (or priority) of operators makes it possible to know how an expression will be evaluated. Indeed, we will start with the operators whose hierarchy is the highest and then we will move on to those whose hierarchy is immediately below. When the hierarchy is the same, the expression is evaluated from left to right.
- ❖ Complex expressions require the use of parentheses "()", because they must be expressed in a linear form (on the same line). Expressions in parentheses are evaluated first and starting with the innermost parentheses.



ELEMENTARY ACTIONS (EXPRESSIONS)

❖ Examples:

❖ Evaluate the following expressions:

➤ $1 + 2 / 3$

➤ $1 + 2 / 3 - 5 * 5 + 3 * 1$

➤ $5 + 2 * 3 * 10 * 10 + 9 * 9$

➤ $(5 + 2 * 3) * (10 * 10 + 9 * 9)$

❖ Write the following expression using algorithmic notation

$$res = \frac{a + \frac{b}{c}}{x + \frac{v + w}{y}}$$

$$res = (a + b / c) / (x + (v + w) / y)$$

ELEMENTARY ACTIONS (EXPRESSIONS)

- ❖ **Logical** expressions are a finite sequence of logical operators relating to Boolean variables or logical values (1 for True, 0 for false).
 - ❖ The logical operators are classified in descending order of priority as follows: NOT (logical negation) then AND (and logical) then OR (or logical).
 - ❖ The notion of true and false varies from one language to another. In C language, the false value is assigned to zero and the true value to any other value ($\neq 0$).

A	B	NOT A	A AND B	A OR B
0	0	1	0	0
0	1	1	0	1
1	0	0	0	1
1	1	0	1	1

ELEMENTARY ACTIONS (EXPRESSIONS)

- ❖ **Relational** expressions are a finite sequence of relational operators ($>$ (*higher*), $\geq$ (*higher or equal*), $<$ (*lower*), $\leq$ (*lower or equal*), $=$ (*equal*), $\neq$ (*different*)) for numeric (integer and real) and alphanumeric (letters and numbers) variables or values.
 - Operators ($<$, $\leq$, $>$, $\geq$) have higher priority than operators ($=$, $\neq$). Operators with the same priority are evaluated from left to right.
- ❖ **Mixed** expressions are a finite sequence of arbitrary operators (arithmetic, logic and relational) relating to arbitrary variables or values (numeric and alphanumeric).
 - Example: $\text{NOT}((x < 5) \text{ OR } (y + z \geq 3))$

ELEMENTARY ACTIONS (EXPRESSIONS)

❖ The evaluation of mixed expressions is generally done from left to right respecting the following rules of priority on the operators (ranked in descending order):

1. The parentheses (we start with the most internal ones).
2. The NOT (LOGICAL), and the $-$ (unary)
3. $*$, $/$, MOD, DIV,
4. $+$, $-$,
5. $<=$, $<$, $>=$, $>$
6. $=$, $\neq$
7. AND
8. OR

➤ Numeric operators with the same priority are evaluated from left to right

ELEMENTARY ACTIONS (EXPRESSIONS)

❖ **Exercise:** Let x, y, z, t be integers. Give the expressions corresponding to the following situations:

1. x and y are both strictly greater than 3
2. x, y and z are the same (equal)
3. x, y and z are the same but different from t
4. Among the values of x, y and z two values and only two are identical
5. x is strictly between y and t (provide two cases)
6. z is not the maximum between the four values.
7. x is even and y is odd
8. z is not a multiple of 3 but it is divisible by 7.
9. x is a multiple of z
10. t is a divisor of x .

ELEMENTARY ACTIONS (EXPRESSIONS)

❖ **Exercise:** Let x, y, z, t be integers.

Situation	Expression
1. x and y are both strictly greater than 3	$(x > 3) \text{ and } (y > 3)$
2. x, y and z are the same (equal)	$(x = y) \text{ and } (y = z)$
3. x, y and z are the same but different from t	$(x = y) \text{ and } (y = z) \text{ and } (z \neq t)$
4. Among the values of x, y and z two values and only two are identical	$((x = y) \text{ and } (x \neq z)) \text{ or } ((x = z) \text{ and } (x \neq y)) \text{ or } ((z = y) \text{ and } (x \neq z))$
5. x is strictly between y and t (provide two cases)	$((y < x) \text{ and } (x < z)) \text{ or } ((z < x) \text{ and } (x < y))$
6. z is not the maximum between the four values.	$(z < x) \text{ or } (z < y) \text{ or } (z < t)$
7. x is even and y is odd	$(x \bmod 2 = 0) \text{ and } (y \bmod 2 \neq 0)$
8. z is not a multiple of 3 but it is divisible by 7.	$(z \bmod 3 \neq 0) \text{ and } (z \bmod 7 = 0)$
9. x is a multiple of z	$x \bmod z = 0$
10. t is a divisor of x .	$x \bmod t = 0$

ELEMENTARY ACTIONS (EXPRESSIONS)

❖ Exercise:

➤ Example: $\text{not } (x=y)$ $\Leftrightarrow$ $x \neq y$ (*x is different from y*)

❖ Based on the previous example, repeat the following expressions without using the "not", then make a translation in the form of sentences:

1. $\text{NOT } ((x < 3) \text{ AND } (y = 5))$
2. $\text{NOT } ((y = x) \text{ OR } (z \neq y))$
3. $\text{NOT } ((x > z) \text{ OR } \text{NOT}(y \leq z))$
4. $\text{NOT } (\text{NOT}(y = z) \text{ OR } \text{NOT}(x = z))$
5. $\text{NOT } ((x \neq y) \text{ AND } (x \neq z) \text{ AND } (y \neq z))$

ELEMENTARY ACTIONS (EXPRESSIONS)

❖ Exercise:

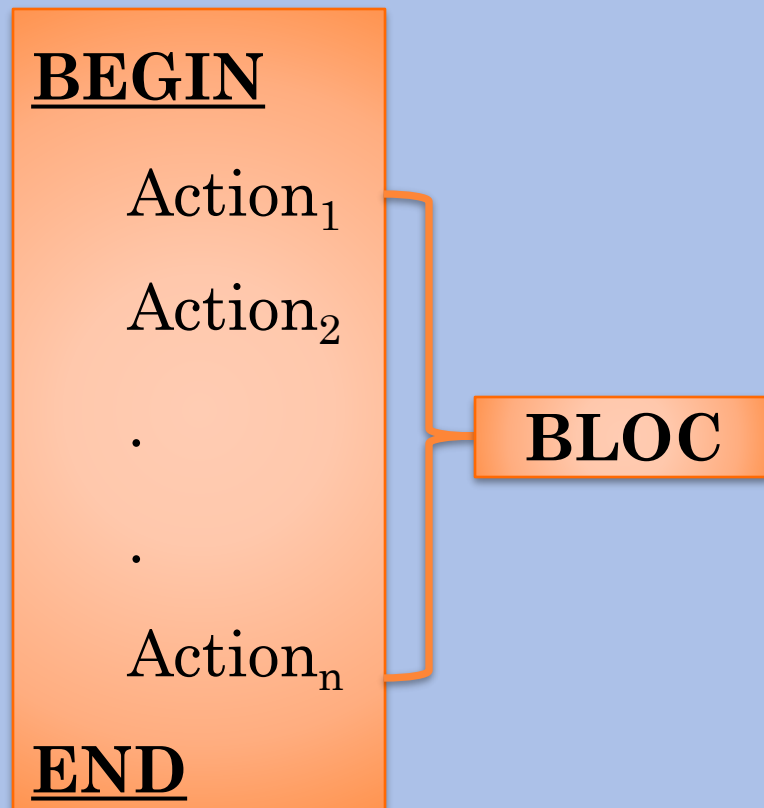
➤ Example: $\text{not } (x=y) \quad \Leftrightarrow \quad x \neq y \quad (x \text{ is different from } y)$

❖ Based on the previous example, repeat the following expressions without using the "not", then make a translation in the form of sentences:

1. $((x \geq 3) \text{ OR } (y \neq 5))$ x greater than or equal to 3 or y different from 5
2. $((y \neq x) \text{ AND } (z=y))$ the values of y and z are identical but different from x
3. $((x \leq z) \text{ AND } (y \leq z))$ z is the maximum of the three values x, y, and z
4. $((y=z) \text{ AND } (x=z))$ the three values x, y, z are equal
5. $((x=y) \text{ OR } (x=z) \text{ OR } (y=z))$ at least two values are equal

CONTROL STRUCTURES

- ❖ **Sequencing.** Primitive actions are executed in the order of appearance in the algorithm. It forms what is called a **block** of actions/instructions.



- **Definition:** A block is a coherent set of one or more actions. It starts with a **BEGIN** of block and ends with an **END** of block. In addition, if a block is composed of a single **primitive (elementary) action**, the begin and end of the block are optional.

CONTROL STRUCTURES (CONDITIONAL)

- ❖ Simple instructions alone are not enough to program everything, some situations require choices and/or repetitions of instructions. We thus need conditional and/or repetitive sequencing.
- ❖ A **conditional sequence** allows certain actions to be executed rather than others, depending on a given condition.
- ❖ A **conditional action** is an instruction that conditions the execution of a sequence of actions on the value of a Boolean expression (condition).
- ❖ Operation :
 1. Evaluation of the condition
 2. If the condition is equal to true then the actions of the $BLOC_{IF}$ are executed,
 3. Proceed sequentially (action after the end of the $BLOC_{IF}$).

```
IF (condition) THEN  
    BEGIN  
         $BLOC_{IF}$   
    END_{IF}
```

CONTROL STRUCTURES (CONDITIONAL)

- ❖ **Example:** Write an algorithm that solves a first degree equation ($a * X + b = 0$)

ALGORITHM Eq_1st_degree

Var a, b: **REAL**

BEGIN

PRINT("Resolution $a*x + b = 0$ ")

PRINT("Introduce a:"); **SCAN**(a)

PRINT("Introduce b:"); **SCAN**(b);

IF ((a=0) **AND** (b=0)) **THEN PRINT**("Indeterminate equation. ")

IF ((a=0) **AND** (b≠0)) **THEN PRINT**("impossibleEquation. ")

IF (a≠0) **THEN PRINT**("The solution of the equation is", $(-1)*b/a$)

End

CONTROL STRUCTURES (ALTERNATIVE)

❖ The **alternative action** is an instruction that makes it possible to condition the execution of a series of actions on the value of a Boolean expression (condition).

❖ Operation :

1. Evaluation of the condition
2. If the condition is true then the actions of BLOCK_{IF} are executed
3. Otherwise (condition equals false) the actions of the $\text{BLOCK}_{\text{ELSE}}$ are executed
4. Proceed sequentially (action after the end of the $\text{BLOC}_{\text{ELSE}}$).

```
IF (condition) THEN
    BEGIN
        BLOCIF
    ENDIF
ELSE
    BEGIN
        BLOCELSE
    ENDELSE
```

- *The conditional is a special case of the alternative*
- *Unable to execute both blocks at once.*

CONTROL STRUCTURES (ALTERNATIVE)

❖ **Exercise:** Write an algorithm that allows:

1. To display the sign of an integer x.
2. To determine the maximum between two integers (using both conditional (a) and alternative (b) forms)
3. To write an algorithm that calculates the absolute value of a given integer, and then indicates whether it is even or odd.

CONTROL STRUCTURES (ALTERNATIVE)

❖ Solution:

1

```
ALGORITHM sign_integer
VAR X: INTEGER
BEGIN
PRINT ("give an integer X = ")
SCAN (X)
IF (X = 0) THEN
    PRINT ("X is nutral ")
ELSE
    IF (X > 0) THEN
        PRINT ("X is positive ")
    ELSE
        PRINT ("X is negative ")
    END
END
```

2(a)

```
ALGORITHM max_cond
Var x, y, max: INTEGER
BEGIN
PRINT ("give two integers: ")
PRINT ("x = "); SCAN (x)
PRINT (" y = "); SCAN (y)
max ← x
IF (y > max) THEN max ← y
PRINT ("max =", max)
END
```

2(b)

```
ALGORITHM max_alter
Var x, y, max: INTEGER
BEGIN
PRINT ("give two integers: ")
PRINT ("x = "); SCAN (x)
PRINT (" y = "); SCAN (y)
IF (x > y) THEN max ← x
ELSE    max ← y
PRINT ("max =", max)
END
```

CONTROL STRUCTURES (ALTERNATIVE)

❖ Solution:

3

ALGORITHM square_absolute_parity

Var x: **INTEGER**

Begin

PRINT ("give an integer x = ")

SCAN (X)

IF (x >= 0) **THEN** **PRINT** ("The absolute value of x is: ", x)

ELSE **PRINT** (" The absolute value of x is: ", (-1) * x)

IF (x MOD 2 = 0) **THEN** **PRINT** ("x is even ")

ELSE **PRINT** ("x is odd ")

END

CONTROL STRUCTURES (ALTERNATIVE)

- ❖ **Example:** Write an algorithm that solves a first degree equation ($a * X + b = 0$)

```
ALGORITHM Eq_1st_degre
```

```
Var a, b: REAL
```

```
BEGIN
```

```
PRINT("Resolution  $Ax + b = 0$ ")
```

```
PRINT("Introduce a:"); SCAN(a)
```

```
PRINT("Introduce b:"); SCAN(b)
```

```
IF (a=0) THEN
```

```
    IF (b=0) THEN
```

```
        PRINT("Indeterminate equation. ")
```

```
    ELSE
```

```
        PRINT(" Impossible Equation. ")
```

```
ELSE PRINT("The solution of the equation is", -b/a)
```

```
End
```

CONTROL STRUCTURES (ALTERNATIVE)

- ❖ The alternative action is a binary choice selective structure (either the BLOCK_{IF} or the BLOCK_{ELSE}). In the case where we want to compare the same variable or expression with several discrete values (not only binary), we must use several nested alternative actions for example:

```
IF (a=1) THEN BEGIN BLOCK1 END
ELSE IF (a=2) THEN BEGIN BLOCK2 END
      ELSE IF (a=4) THEN BEGIN BLOCK3 END
            ELSE IF (a=8) THEN BEGIN BLOCK4 END
      ELSE BEGIN BLOCK5 END
END
```

```
SWITCH a
BEGIN
CASE 1 : BEGIN      BLOC1      END
CASE 2 : BEGIN      BLOC2      END
CASE 4 : BEGIN      BLOC3      END
CASE 8 : BEGIN      BLOC4      END
DEFAULT: BEGIN      BLOC5      END
ENDSWITCH
```

- ❖ This suite can be replaced with the multiple choice action SWITCH

CONTROL STRUCTURES (ALTERNATIVE)

- ❖ The multiple choice action makes it possible to perform a series of consecutive equality tests for a given value and to execute instructions according to the value.

```
SWITCH (Expression)
BEGIN
CASE constant1 : BEGIN      BLOC1      END
CASE constant2 : BEGIN      BLOC2      END
.
.
CASE constantn : BEGIN      BLOCn      END
[DEFAULT:      BEGIN      BLOCdefault  END] //optional
ENDSWITCH
```

- ❖ The **DEFAULT** block allows to define the actions to be performed if the value of the expression does not correspond to any constant. It is optional (you can not put it).

CONTROL STRUCTURES (ALTERNATIVE)

- ❖ **Example:** Write an algorithm that reads the number of a given month and then displays the number of days in the month.

```
ALGORITHM number_days_months_alter
Var m: INTEGER
BEGIN
PRINT("Give month number between 1 and 12:"); SCAN(m)
IF ((m=1) OR (m=3) OR (m=5) OR (m=7) OR (m=8) OR (m=10) OR (m=12)) THEN
    PRINT("This month contains 31 days ")
ELSE IF ((m=4) OR (m=6) OR (m=9) OR (m=11)) THEN
    PRINT("This month contains 30 days ")
ELSE IF (m=2)      PRINT ("This month contains 28 or 29 days ")
ELSE              PRINT ("ERROR: invalid month")

End
```

CONTROL STRUCTURES (ALTERNATIVE)

- ❖ **Example:** Write an algorithm that reads the number of a given month and then displays the number of days in the month.

```
ALGORITHM number_days_months_switch
Var m: INTEGER
BEGIN
PRINT("Give month number between 1 and 12:"); SCAN(m)
SWITCH m
BEGIN
CASE 1, 3, 5, 7, 8, 10, 12:    PRINT("This month contains 31 days ")
CASE 4, 6, 9, 11:            PRINT("This month contains 30 days ")
CASE 2:                      PRINT ("This month contains 28 or 29 days ")
DEFAULT:                     PRINT ("ERROR: invalid month")
END_SWITCH
END
```

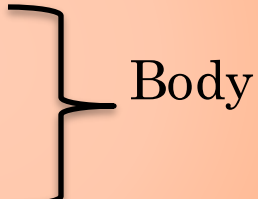
CONTROL STRUCTURES (LOOP)

- ❖ Many treatments require repeating a block several times, with the possibility of varying certain parameters.
- ❖ A **Loop** makes it possible to execute a block of instructions several times according to a condition.
- ❖ Each execution of the block is called **an iteration**.
- ❖ There are 3 forms of repetitions (loops): FOR, WHILE .. DO and DO .. WHILE.
 - When the number of iterations is known in advance, we often use the "FOR" loop, otherwise (unknown) we must use the other LOOPS (WHILE).
 - When the loop body has to be executed at least once (from 1 to several times), it is advisable to use the DO .. WHILE loop.
 - When the loop body may not be executed (from 0 to several times), it is advisable to use the WHILE .. DO loop .

CONTROL STRUCTURES (LOOP)

- ❖ The general shape of the For loop is as follows:

```
FOR  $CT \leftarrow IV$  TO  $FV$  [step  $p$ ] DO  
BEGIN  
BLOCK  
ENDFOR
```



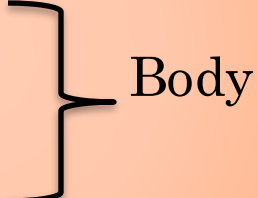
A right-facing curly bracket is positioned to the right of the code block, spanning the lines from **BEGIN** to **END**_{FOR}. The word "Body" is written to the right of the bracket, indicating the code between **BEGIN** and **END**_{FOR} constitutes the loop's body.

- The control variable (CT) is used to enumerate the iterations. It is called the loop counter.
- IV , FV and p : can be integers, integer type variables, or integer value expressions.
- The For loop is used when the number of repetitions is known in advance, or when the initial value (IV) and the final value of the counter (FV) are known.
- The counter will successively take the values between IV and FV by adding to it each time the value of the step p .
- For each value taken by CT , the actions of the block are executed.

CONTROL STRUCTURES (FOR)

- ❖ The general shape of the For loop is as follows:

```
FOR  $CT \leftarrow IV$  TO  $FV$  [step  $p$ ] DO  
BEGIN  
BLOCK  
ENDFOR
```

A large right-facing curly bracket is positioned to the right of the code block, spanning from the line 'BEGIN' to the line 'END_FOR'. The word 'Body' is written to the right of the middle of this bracket, indicating that the code between BEGIN and END_FOR constitutes the body of the loop.

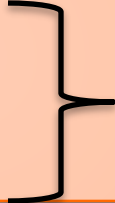
- ❖ Operations in the event that the step is positive[**case 2: $p < 0$**]

1. initialization: $CT \leftarrow IV$,
2. Evaluation of the stop condition: condition ($CT \leq FV$) [$CT \geq FV$]
3. If condition = true then
 - Body instructions (BLOCK) are executed,
 - $CT \leftarrow CT + p$
 - Back to Step 2
4. Otherwise (condition = false): end of the loop and continuation after the

CONTROL STRUCTURES (FOR)

- ❖ The general shape of the For loop is as follows:

```
FOR  $CT \leftarrow IV$  TO  $FV$  [step  $p$ ] DO  
BEGIN  
BLOCK  
ENDFOR
```



Body

- ❖ **STEP** is optional; and If they are not set [**FOR** $CT \leftarrow IV$ **TO** FV **DO**] then by default the CT is incremented by one ($p = 1$)
- ❖ The block is repeated $((|FV - IV|) \div p + 1)$ times.

CONTROL STRUCTURES (FOR)

❖ **Example:** Unroll the following loops:

FOR i ← 0 TO 50 step 10 DO
 print(i)

<u>Ecran</u>
0
10
20
30
40
50

FOR j ← 30 TO 5 step -5 DO
 PRINT(j)

<u>Ecran</u>
30
25
20
15
10
5

FOR i ← 1 TO 3 DO
 FOR j ← 1 TO 2 DO
 PRINT("i = ", i, " j = ", j)

<u>Ecran</u>
i=1 j=1
i=1 j=2
i=2 j=1
i=2 j=2
i=3 j=1
i=3 j=2

CONTROL STRUCTURES (FOR)

❖ **Exercise:** Write an algorithm that displays

1. the sum of integers that are between 1 and 100.
2. all natural numbers multiples of 5 that are between 1 and 100
3. All divisors of an integer N.

1

```
ALGORITHM sum1
Var i, S: INTEGER
BEGIN
S ← 0
FOR i ← 1 TO 100 DO
    S ← S + i
PRINT ("Sum S =", S)
END
```

2 (v1)

```
ALGORITHM sum2_v1
Var i: INTEGER
BEGIN
FOR i ← 5 TO 100 step 5 DO
    PRINT (i)
END
```

2 (v2)

```
ALGORITHM som2_v2
Var i: INTEGER
BEGIN
FOR i ← 1 TO 20 DO
    PRINT (5*i)
END
```

CONTROL STRUCTURES (FOR)

❖ **Exercise:** Write an algorithm that displays

3. All divisors of an integer N.

ALGORITHM DIVIDERS

Var N, i: Integer

BEGIN

PRINT (" Give strictly positive integer N = ")

SCAN (N)

PRINT ("The list of its divisors: 1 , ")

FOR $i \leftarrow 2$ **TO** (N div 2) **DO**

IF ((N MOD i) = 0) **THEN PRINT** (i, " , ")

PRINT(N, " .")

END

CONTROL STRUCTURES (WHILE .. Do)

❖ The shape of the " while Do" loop is as follows:

- ❖ the condition must be computable (before any iteration)
- ❖ the body of the loop may never be executed,
- ❖ the body must modify the variables of the condition (to avoid an infinite loop)

```
WHILE (Condition) DO  
  BEGIN  
    BLOCK  
  ENDWHILE
```

} Body

❖ Operations:

1. Condition Assessment
2. If condition = true then
 - Body instructions (BLOCK) are executed,
 - Go back to Step 1
3. Otherwise (condition = false): end of the loop and continuation after the
END_{while}.

CONTROL STRUCTURES (Do .. WHILE)

❖ The shape of the "do While" loop is as follows:

- The two forms are equivalent: BEGIN – END are optional even if the block has multiple instructions.
- the condition must be computable (after any iteration),
- the body of the loop is executed **at least once**
- the body must modify the variables of the condition
- (to avoid an infinite loop)

❖ Operations:

1. Body instructions (BLOCK) are executed,
2. Condition Assessment
3. If condition = true then return to step 1
4. Otherwise (condition = false): end of the loop and continuation after the WHILE.

Shape 1

```
DO
  Begin
    BLOCK } Body
END
WHILE (Condition)
```

Shape 2

```
DO
  BLOCK
WHILE (Condition)
```

CONTROL STRUCTURES

- ❖ **Exercise:** Translate the following “For” loop into “do ... while” and “while ... do” loops

FOR i←0 TO 50 STEP 10 DO PRINT(i)	FOR j←30 TO 5 STEP -5 DO PRINT(j)	FOR i←1 TO 3 DO FOR j←1 TO 2 DO PRINT("i = ", i, " j = ", j)
i←0 WHILE (i≤50) DO BEGIN PRINT(i) i←i+10 END	j←30 WHILE (j≥5) DO BEGIN PRINT(j) j←j-5 END	i←1 WHILE (i≤3) DO BEGIN j←1 WHILE (j≤2) DO BEGIN PRINT("i = ", i, " j = ", j) j←j+1 END i←i+1 END

CONTROL STRUCTURES

- ❖ **Exercise:** Translate the following “For” loop into “do ... while” and “while ... do” loops

FOR i←0 TO 50 STEP 10 DO PRINT(i)	FOR j←30 TO 5 STEP -5 DO PRINT(j)	FOR i←1 TO 3 DO FOR j←1 TO 2 DO PRINT("i = ", i, " j = ", j)
i←0 DO BEGIN PRINT(i) i←i+10 END WHILE (i≤50)	j←30 DO BEGIN PRINT(j) j←j-5 END WHILE (j≥5)	i←1 DO BEGIN j←1 DO BEGIN PRINT("i = ", i, " j = ", j) j←j+1 END; WHILE (j≤2) i←i+1 END; WHILE (i≤3)

CONTROL STRUCTURES

❖ **Exercise:** Write an algorithm that allows to

1. read a positive integer x , then double it as many times until it exceeds 60.
2. display the sum of n integers ($n > 0$) as well as their average.
3. display the minimum between n integers ($n > 0$).

CONTROL STRUCTURES

- ❖ **Exercise:** (1) Write an algorithm that reads a positive integer x , then double it as many times until it exceeds 60.

```
ALGORITHM double60
```

```
VAR  $x$  : INTEGER;
```

```
BEGIN
```

```
DO
```

```
    PRINT(" Give Integer >0 :");
```

```
    SCAN( $x$ )
```

```
WHILE ( $x \leq 0$ )
```

```
WHILE ( $x \leq 60$ ) DO  $x \leftarrow x * 2$  PRINT ("  $x =$  ",  $x$ ) // equivalent to For  $x$  to 60 step  $x$  do
```

```
END
```

CONTROL STRUCTURES

- ❖ **Exercise:** (2) Write an algorithm that displays the sum of n integers ($n > 0$) as well as their average.

```
ALGORITHM sum_avg
VAR n, i, x, S: INTEGER; A : REAL
BEGIN
    DO
        PRINT(" Give  $n > 0$ :"); SCAN(n)
    WHILE ( $n \leq 0$ )
     $S \leftarrow 0$ 
    FOR  $i \leftarrow 1$  TO n DO
        BEGIN
            PRINT("Give Integer:"); SCAN(x)
             $S \leftarrow S + x$ 
        END
     $A \leftarrow S / n$ 
    PRINT (" Sum = ", S, " Average= ", A)
END
```

CONTROL STRUCTURES

- ❖ **Exercise:** (3) Write an algorithm that displays the minimum between n integers ($n > 0$).

```
ALGORITHM min_N_integer  
VAR n, i, min: INTEGER;  
BEGIN  
    DO  
        PRINT(" Give  $n > 0$ :"); SCAN(n)  
    WHILE ( $n \leq 0$ )  
        PRINT("Give Integer:"); SCAN(min)  
    FOR  $i \leftarrow 2$  TO n DO  
        BEGIN  
            PRINT("Give Integer:"); SCAN(x)  
            IF ( $x < \text{min}$ ) THEN  $\text{min} \leftarrow x$   
        END  
    PRINT (" MIN = ", min)  
END
```

CONTROL STRUCTURES (NESTING)

- ❖ Control structures are said to be **nested** when one contains the other.
- ❖ Several nestings are then possible:
 - **tests in tests,**
 - tests in loops,
 - loops in tests,
 - **loops in loops,**
 - tests in loops in tests,
 - etc

CONTROL STRUCTURES (NESTING)

- ❖ Rule 1: The **ELSE** always refers to the last **IF** encountered.
 - Good use of **tabs (right offsets or "indentation")** is essential to make the program **readable**, especially with nested if-else.
 - Give the execution result of this form.

Shape 1

```
IF (C1) THEN B1
ELSE IF (C2) THEN B2
ELSE IF (C3) THEN B3
ELSE Bother
```

Shape 2

```
IF (C1) THEN B1
|
ELSE IF (C2) THEN B2
|
ELSE IF (C3) THEN B3
|
ELSE Bother
```

Shape 1 Conditions	Block to execute
C1 = T	B1
C1 = F et C2 = T	B2
C1 = F et C2 = F et C3 = T	B3
C1 = F et C2 = F et C3 = F	Bother

CONTROL STRUCTURES (NESTING)

- ❖ Rule 1: The **ELSE** always refers to the last **IF** encountered.
 - Give the result of execution of the following forms:

Shape 3

```
IF (C1) THEN
    IF (C2) THEN B2
    ELSE B1
ELSE IF (C3) THEN B3
ELSE Bother
```

Shape 3 Conditions	Block to execute
C1 = T et C2 = T	B2
C1 = T et C2 = F	B1
C1 = F et C3 = T	B3
C1 = F et C3 = F	Bother

Shape 4

```
IF (C1) THEN
    IF (C2) THEN B2
ELSE IF (C3) THEN B3
ELSE Bother
```

Shape 4 Conditions	Block to execute
C1 = T et C2 = T	B2
C1 = T et C2 = F et C3 = T	B3
C1 = T et C2 = F et C3 = F	Bother

CONTROL STRUCTURES (NESTING)

- ❖ Rule 1: The **ELSE** always refers to the last **IF** encountered.
 - How can shape 4 be modified in order to obtain the following execution result:

Shape 4

```
IF (C1) THEN
    IF (C2) THEN B2
ELSE IF (C3) THEN B3
ELSE Bother
```

Shape 5 Conditions	Bloc à exécuter
C1 = T et C2 = T	B2
C1 = F et C3 = T	B3
C1 = F et C3 = F	Bautre

Shape 5

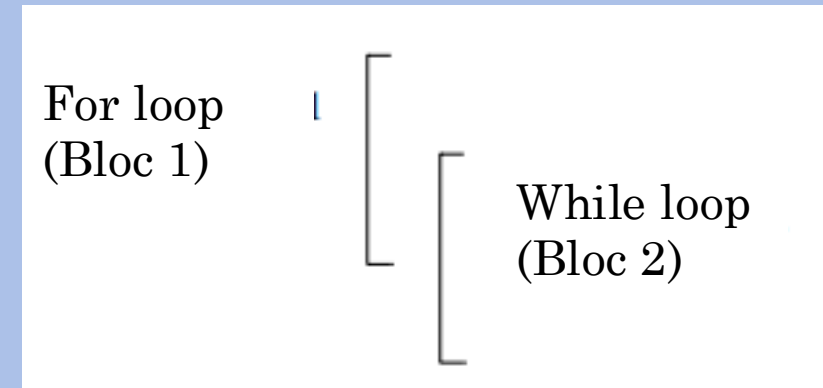
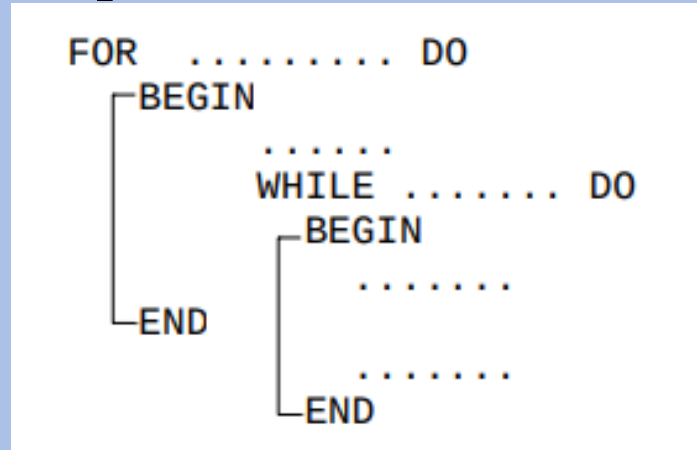
```
IF (C1) THEN
    BEGIN
        IF (C2) THEN B2
    END_IF
ELSE IF (C3) THEN B3
ELSE Bother
```

- The solution is to **protect** "if C2 then B2" in a **BEGIN - END_{IF}**.
- ❖ Rule 2: Indicate between **BEGIN - END_{IF}** the specific case in which you are.

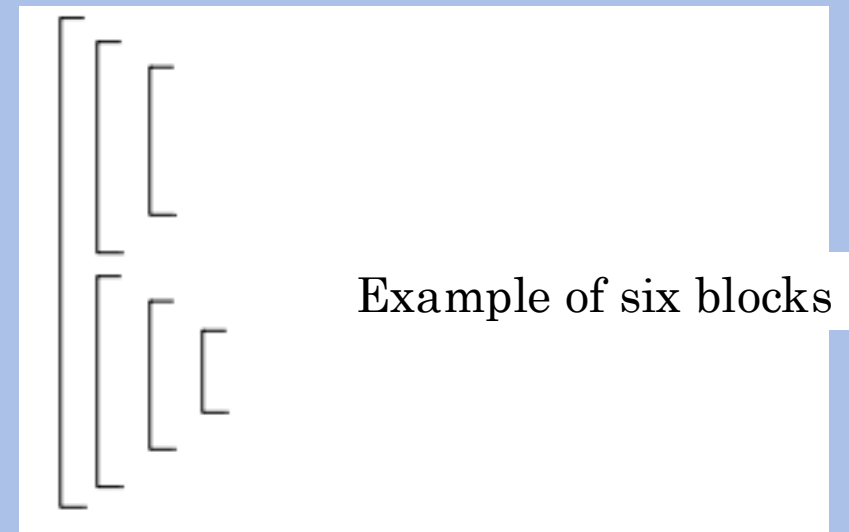
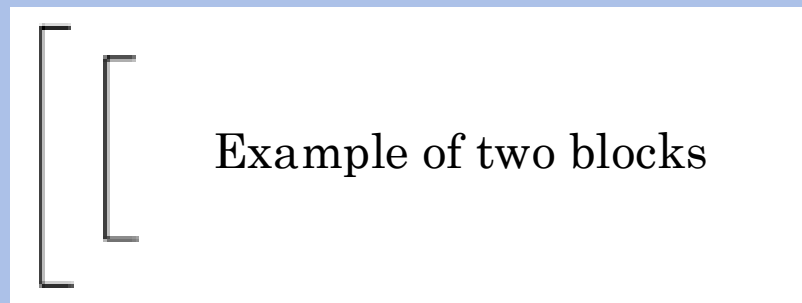
CONTROL STRUCTURES (NESTING)

❖ Rule 3: Overlapping is prohibited,

➤ Example:

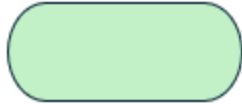
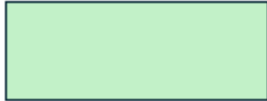
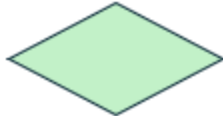


➤ Solution: Close internal blocks before external blocks,



FLOWCHART

- ❖ A flowchart is a graphical representation of how a problem is solved. It is similar to an algorithm. Each type of action in the algorithm has a representation in the flowchart.
- ❖ Symbols used in flowcharts:

Symbol	Name	Function
	Oval	Represents the start or end of a process
	Rectangle	Denotes a process or operation step
	Arrow	Indicates the flow between steps
	Diamond	Signifies a point requiring a yes/no
	Parallelogram	Used for input or output operations

FLOWCHART

❖ Representation of algorithmic primitives:

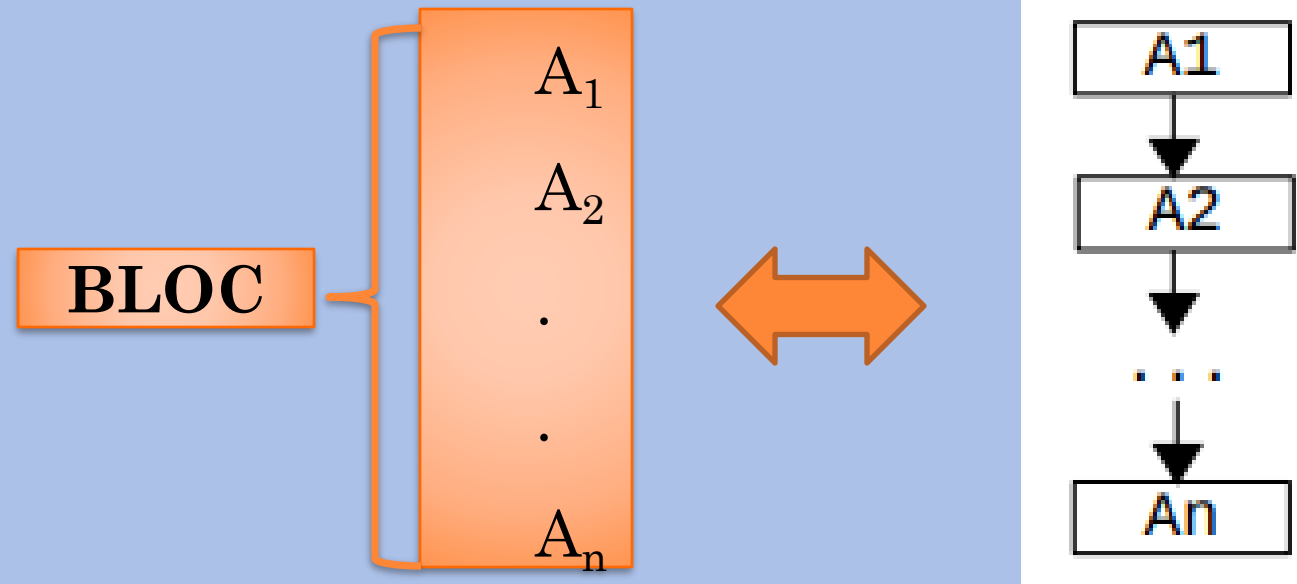
➤ Reading

READ / SCAN ([f], Var₁, Var₂, ... , Var_n)

➤ Writing

WRITE / PRINT ([f], R₁, R₂, ... , R_n)

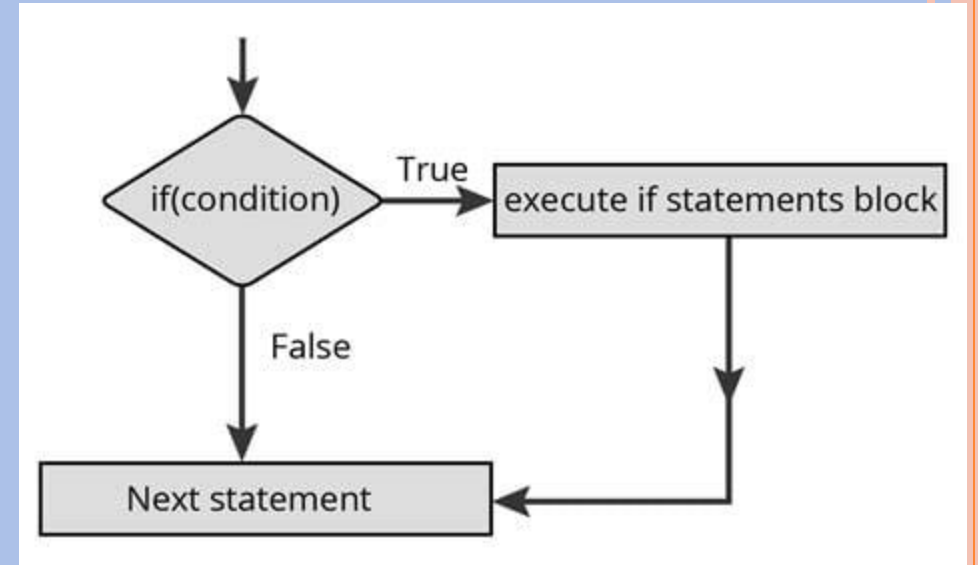
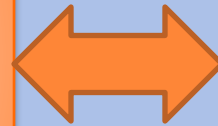
➤ Sequence: A: primitive action (assignment)



FLOWCHART

- ❖ Representation of algorithmic primitives:
- ❖ Conditional

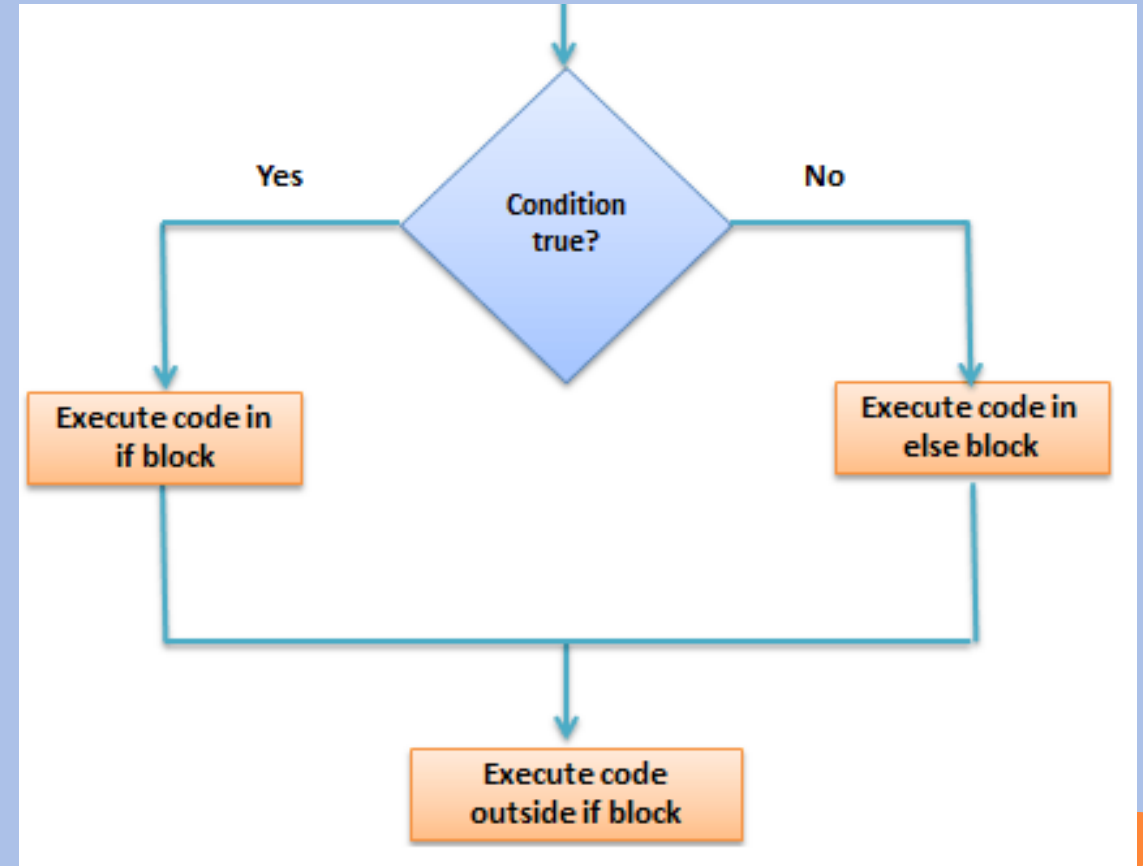
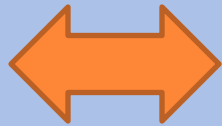
IF (conditions) **THEN**
BEGIN
Action(s)
END



FLOWCHART

- ❖ Representation of algorithmic primitives:
- ❖ Alternative

```
IF (condition) THEN
    BEGIN
        Action1(s)
    END
ELSE
    BEGIN
        Action2(s)
    END
```

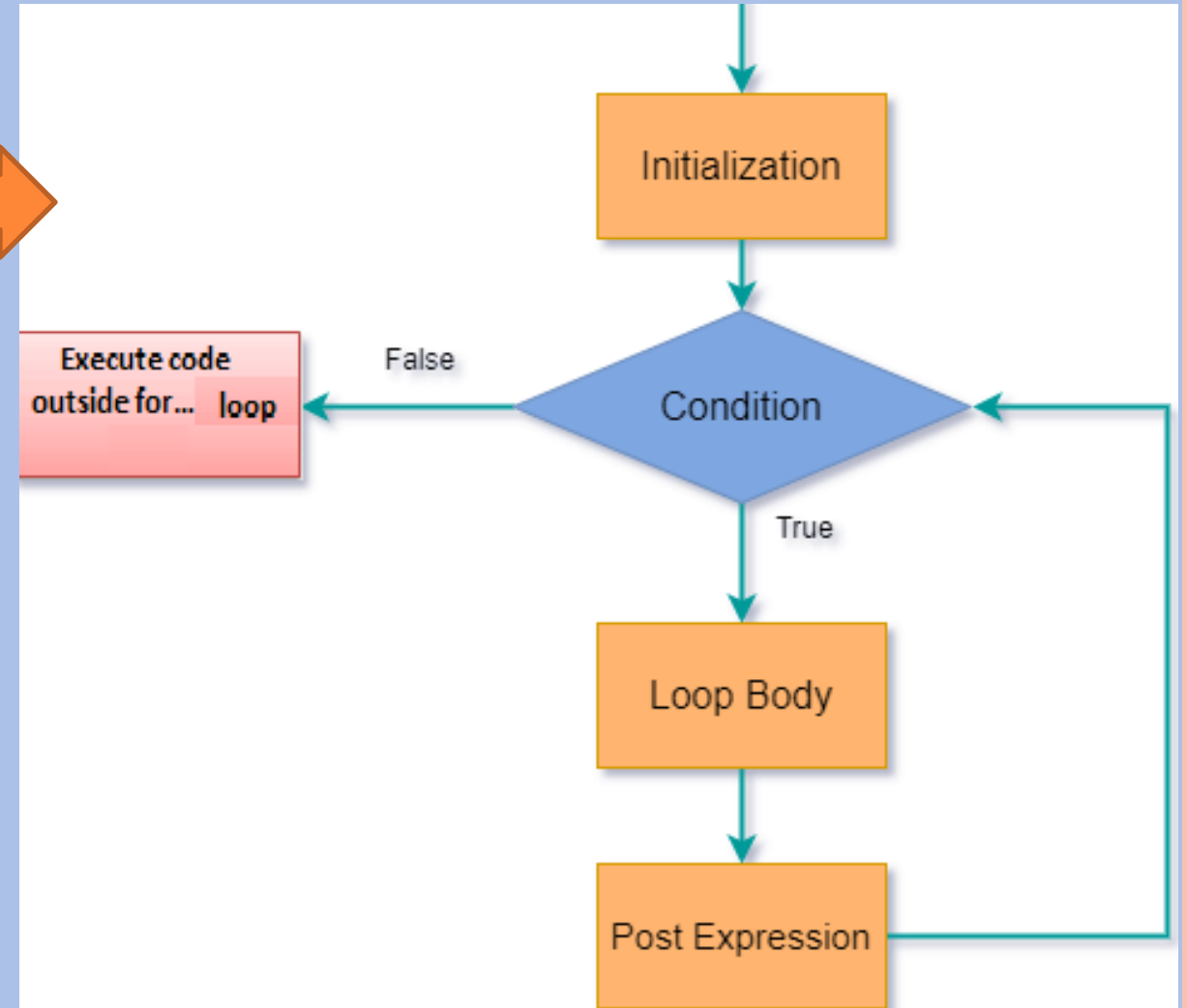
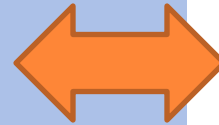


FLOWCHART

❖ Representation of algorithmic primitives:

❖ “For” Loop

```
FOR  $CT \leftarrow IV$  TO  $FV$  DO  
  BEGIN  
    Action(s)  
  END
```

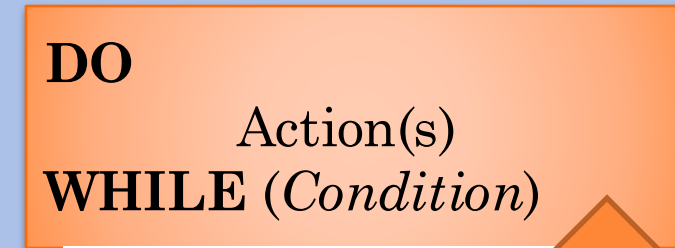
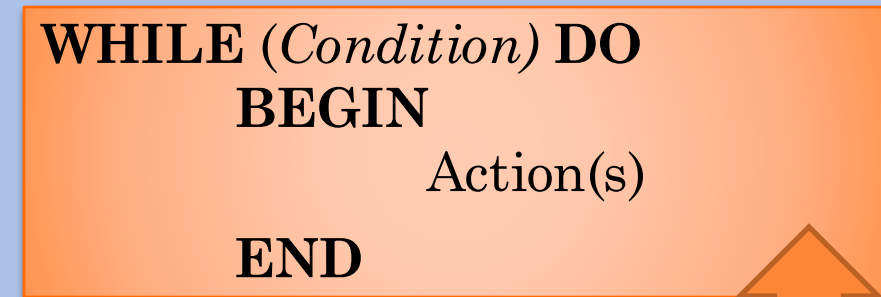


FLOWCHART

❖ Representation of algorithmic primitives:

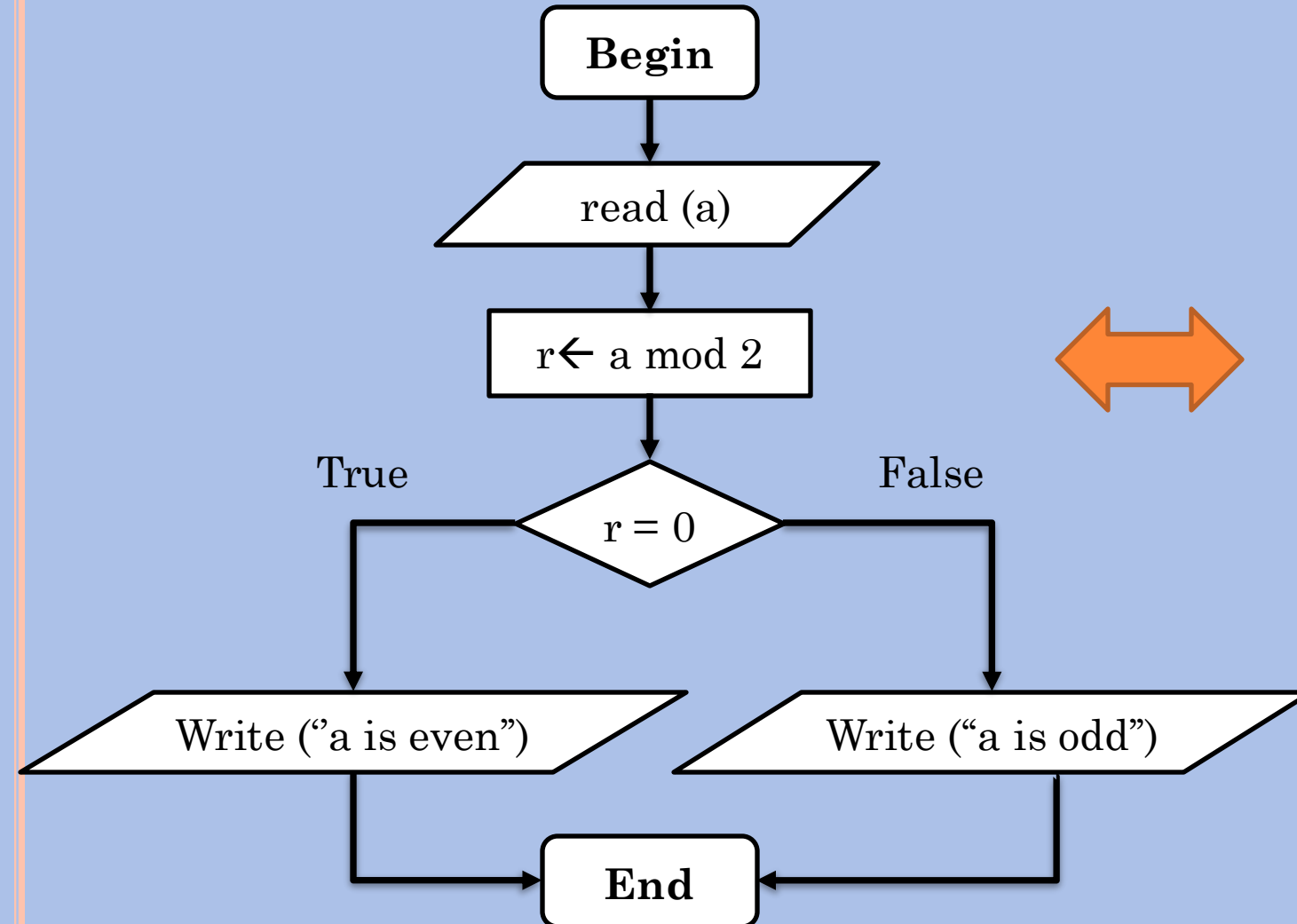
❖ “while .. Do “Loop

“do ... while “ Loop



FLOWCHART

❖ **Example:** Give the algorithm corresponding to the following flowchart:



Algorithm parity
var a,r:integer
BEGIN
print (" Give value of a: ")
scan (a)
 $r \leftarrow a \bmod 2$
if (r = 0) then print("a is even ")
else print("a is odd")
END

FLOWCHART

❖ **Example:** give the algorithm corresponding to the following flowchart:

➤ What does this algorithm do?

Algorithme exemple;

var

n,m,a : integer

BEGIN

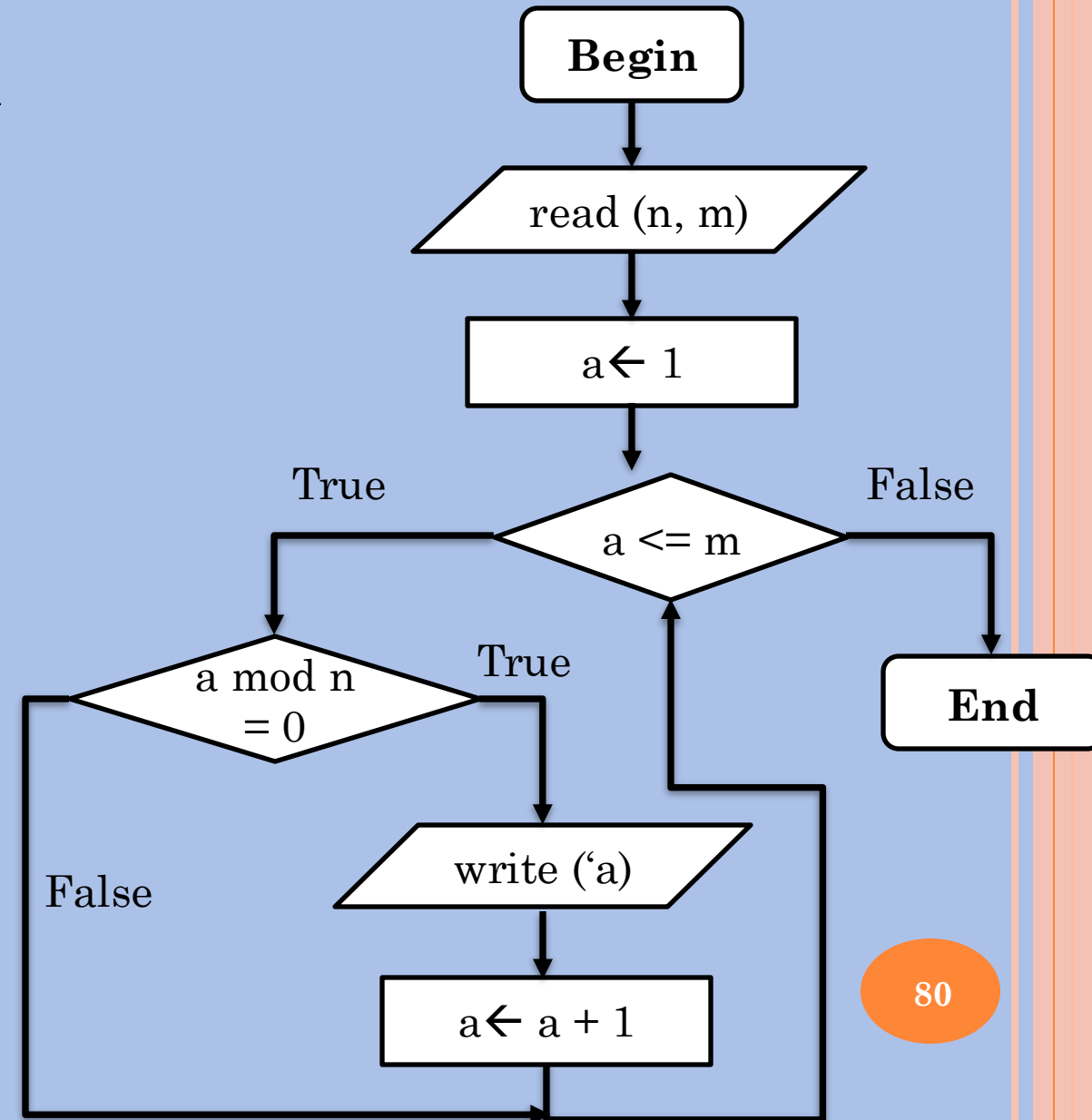
print (" Give the value of n and m: ");

scan (n, m);

for a←1 to m do

if (a mod n = 0) then print (a)

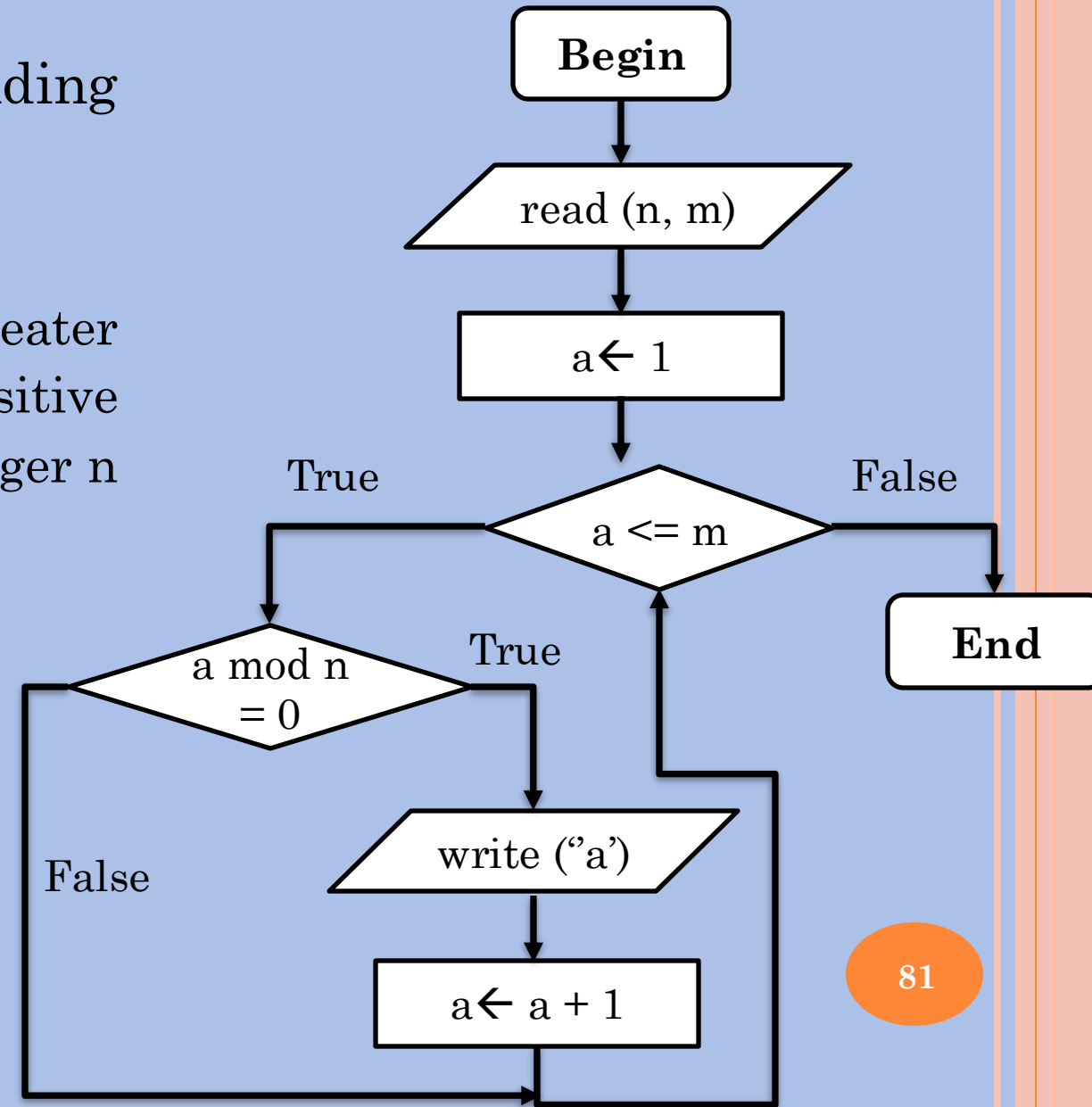
END



FLOWCHART

❖ **Example:** give the algorithm corresponding to the following flowchart:

- What does this algorithm do?
- ✓ It is used to display all integers greater than or equal to 1 and less than m (positive integer) and which are divisible over an integer n .



FLOWCHART

- ❖ It is better to use algorithmic representation than flowchart representation especially when the problem is complex because it becomes more difficult to read and understand than an algorithm. Indeed, the flowchart will be long and will last for more than one page. In addition, there is a problem of overlapping arrows.

COURSE SOURCES

- B. CHERGOU (2014), N. MEDJAOUI (2018) & N. BERREHOUMA, *Cours d'algorithmique/pascal*, 1st-year CPI, Higher National School of Computer Science (ESI) Oued Smar, Algiers.
- M. ZAIR (2022), *Algorithmique 1*, 1st-year License Common Core MI (Mathematics and Computer Science), Faculty of Sciences, Saad Dahlab University, Blida.
- O. Hadj Yahia (2022), *Bureautique, algorithmique et langage de programmation*, 1st-year License Common Core SM, Faculty of Sciences, Saad Dahlab University, Blida.
- R. OUZEGGANE (2023), Computer Science, ING ST (INGénieur Sciences et Technologies), Bejaia University.



Saad Dahlab University - Blida1

Faculty of Sciences

Department of Computer Science

Computer Science Engineering

Semester 1 (1st year)

Algorithmic and Data Structures 1



CHAPTER III: ARRAYS

Dr. MUSTAPHA CHABA MOUNA(chabamouna_mustapha@univ-blida.dz)

2025-2026

CONTENT

- ❑ Introduction
- ❑ Definitions
- ❑ Vectors.
- ❑ Strings
- ❑ Matrixes

INTRODUCTION

- ❖ Some situations cannot be treated with simple variables only. **They can only keep one value at a time.**
- ❖ Using multiple data multiple times can be tricky in a program.
 - Example: Rank the students in the group (30 students!) according to their average □ use 30 real variables!!

```
Var M1 , M2 ,M3 ,M4 ,M5 ,M6 ,M7 ,M8 ,M9 ,M10 ,M11 ,M12 ,M13 ,.....: real
```

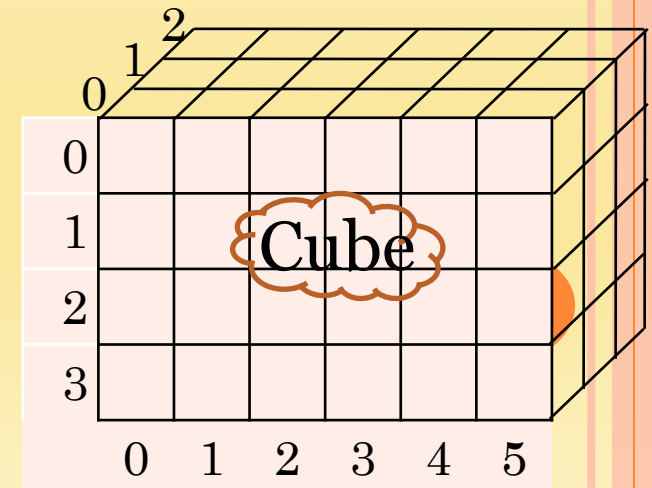
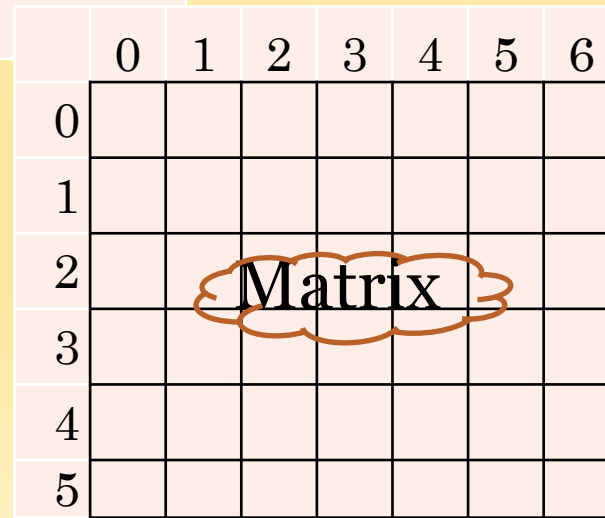
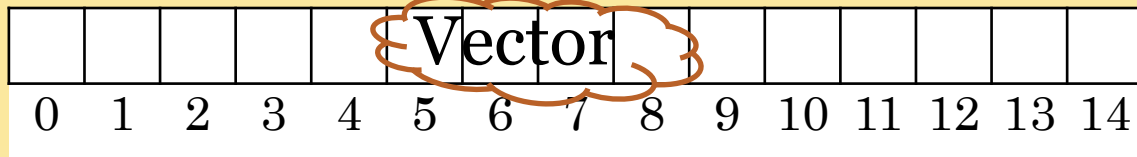
- **Arrays: allow to regroup the averages into a structure.**

M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12	...	...	M30
----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----

DEFINITIONS

- ❖ **Arrays** are structured variables, consisting of a fixed number (static) of elements (items), **all of the same type**, and accessible using indexes. They are characterized by their dimension:

Dimension	Appellation
1	One-dimensional array or Vector
2	Two-Dimensional array or Matrix
3	Three-dimensional array or Cube
$N > 3$	multidimensional array



DEFINITIONS

- ❖ To define an array, you should specify:
 - its dimension
 - the maximum number of its elements in each dimension (i.e. d), i.e. its maximum size in each dimension (i.e. $Smax_{dimension}$)
 - the type of each element (*typelt*)
 - its name (*identifier*)
- ❖ The declaration is made by specifying the word **ARRAY**, followed by the size in square brackets of each dimension followed by the word **OF** and the type of elements.

*identifier: ARRAY [$Smax_1$] **[** $Smax_2$ **]**.... $Smax_d$ **]** OF *typelt**

The number of brackets [] designates the **dimension (d)** of the array.

DEFINITIONS

❖ Example:

Dimension	Algorithmic Language (AL)	C Language (CL)
1: <i>Vector</i>	VAR V: array [100] of integer	int V [100];
2: <i>Matrix</i>	VAR M: array [100][200] of integer	int M [100][200];
3: <i>Cube</i>	VAR C: array [100][200][300] of integer	int C [100][200][300];

- V is a vector of 100 elements,
- M is a matrix of 100 rows and 200 columns (each row is a vector)
- C is a cube of 100 layers where each layer has 200 rows and 300 columns (each layer is a matrix).

DEFINITIONS

- ❖ There are other methods of declaring the array:

Method No	Algorithmic language	C language
<i>1: declare as much as variable</i>	VAR A: array [100] of integer	int A [100];
<i>2: use a constant to declare the maximum size (Smax)</i>	CONST Smax = 100 VAR A: array [Smax] of integer	const int Smax = 100; int A [Smax];
<i>3: declare as much as a new type</i>	CONST Smax = 100 TYPE TAB:array [Smax] of integer VAR A : TAB	const int Smax = 100; typedef int TAB [Smax]; TAB A;

DEFINITIONS

- ❖ In static arrays, a distinction is often made between the maximum size (or declared size) and the real size (or used size) of the arrays.

	maximum (declared) size	real (used) size
Definition	It represents the total capacity of the array, meaning, the maximum number of elements that the array can store.	It represents the number of elements actually used in the array at a given time.
Declaration and modification	It is determined at the time of declaration and remains fixed throughout the life of the array.	It is determined during execution (usually by a reading) and may vary depending on the applicative logic.
Remark	The size should not be too large to avoid wasting memory.	It must be less than or equal to the maximum size of the table.

DEFINITIONS

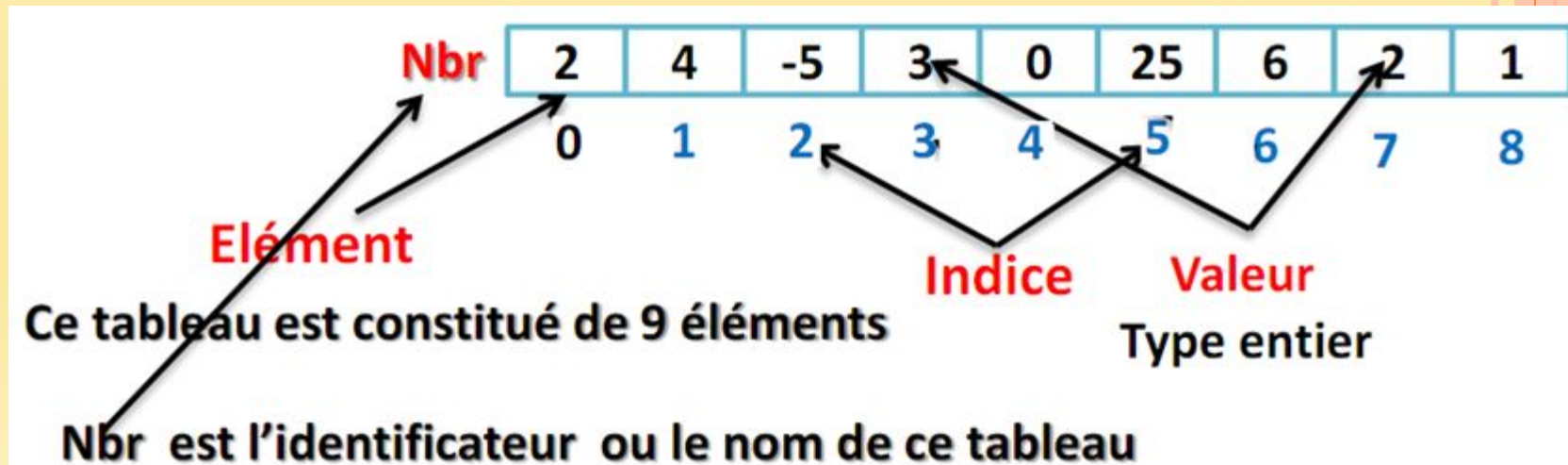
- ❖ Access to an element of the array is done by specifying the name of the array followed by the value of the index in square brackets for each dimension:

identifiant [**index₁**] *[[index₂]]... [index₃]]*

- with **index_d**: a constant, a variable or an integer expression whose value must be between 0 and (**Smax_d** - 1).

- ❖ **EXAMPLE 1 (Vector):**

- Nbr: array [9] of integer
- The third element is in position 2.



DEFINITIONS

❖ EXAMPLE 1 (Vector):

❖ A COEF array containing the coefficients of 7 courses

❑ **COEF: array [7] of integer**

6	4	5	3	2	2	1
0	1	2	3	4	5	6

❑ The following actions can be written

$A \leftarrow \text{COEF}[3]$; $B \leftarrow \text{COEF}[k]$; $C \leftarrow \text{COEF}[(\text{test}+k) \bmod i]$

$\text{COEF}[6] \leftarrow E$; $\text{COEF}[m] \leftarrow F$; $\text{COEF}[a * \text{res} \bmod 2] \leftarrow G$

❑ It is noted that an element of the array is considered as a variable and that the value of the index can be a constant, a variable or an expression whose value must be between 0 and 6 ($S_{\max}-1$).

USUAL VECTOR OPERATIONS

❖ The usual operations on **vectors** (or one-dimensional arrays) include :

1. Vector Initialization

- Empty initialization: create a vector without initial values.
- Initialization with given values, for example, values read from the keyboard (**reading elements of a vector**).
- Initialization with a default value (zeros or others).

USUAL VECTOR OPERATIONS

- ❖ **Note 1 (Initialization):** In C language, static arrays can be **initialized when they are declared**. The initialization can be done by specifying the values between the brackets { } when declaring. If values are not specified, the array items are initialized with default values (0 for numeric types).

Declaration	Array Image								
int T1[3] = {8, 1, 12};	<table><tr><td>8</td><td>1</td><td>12</td></tr></table>	8	1	12					
8	1	12							
int T2[8] = {1, 5, 1}									
int T3[] = {10, 5, 8, -1};	<table><tr><td>1</td><td>5</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	1	5	1	0	0	0	0	0
1	5	1	0	0	0	0	0		
	<table><tr><td>10</td><td>5</td><td>8</td><td>-1</td></tr></table>	10	5	8	-1				
10	5	8	-1						

USUAL VECTOR OPERATIONS

❖ Note 1 (Initialization):

- It is always possible to initialize an array using a loop in the body of the main program or module.
- For example, the array T can be initialized in different ways:

Initialization	Langage Algorithmique (LA) T : array [20] of integer	Langage C (LC) int T [20];
with a single constant (c)	for i□0 to 19 do T[i]□c;	for (i = 0 ; i<20 ; i++) T[i]=c;
with different constants	for i□0 to 19 do T[i] □i;	for (i = 0 ; i<20 ; i++) T[i]=i;
with reading values	for i□0 to 19 do scan(T[i]);	for (i = 0 ; i<20 ; i++) scanf("%d", &T[i]);

USUAL VECTOR OPERATIONS

❖ The usual operations on **vectors** (or one-dimensional arrays) include :

1. Vector Initialization

- Empty initialization: create a vector without initial values.
- Initialization with given values, for example, values read from the keyboard (**reading elements of a vector**).
- Initialization with a default value (zeros or others).

2. Browse

- Browse the vector to display, modify or perform calculations on elements.
- Copy one vector to another.

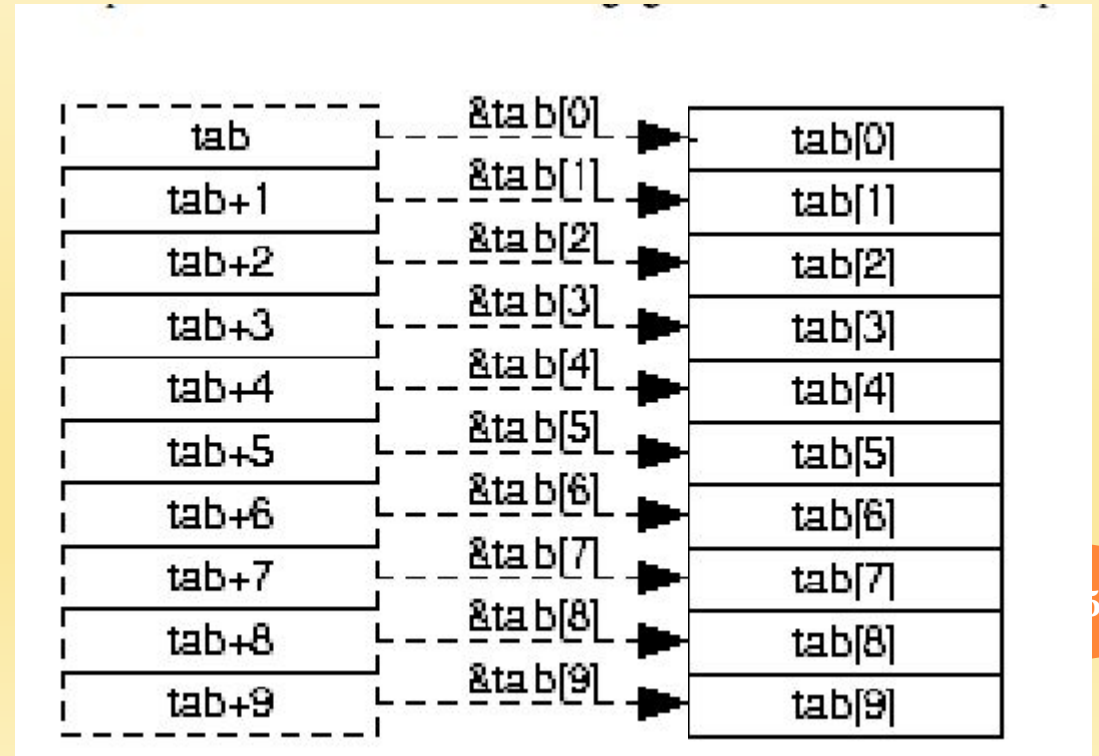
USUAL VECTOR OPERATIONS

❖ Note 2 (Assigning one array to another)

- The declaration of an array reserves space in memory for the elements of the array and provides the address of the first element of the array in the name of the array (without brackets).

□ Example:

tab: array[10] of integer



USUAL VECTOR OPERATIONS

- ❖ **Note 2 (Assigning one array to another)**
- ❖ The name of the array actually matches the address of the first item in the array. This implies in particular that no global operations are allowed on a array. **In particular, an array cannot appear to the left of an assignment operator.**
- For example, **“T1□T2;” cannot be written.** The assignment must be made for each of the elements of the table using a loop:

Copy one array to another	Algorithmic Language (AL)	C Language (CL)
	Const N = 50 T1, T2 : array [N] of integer for i□0 to N-1 do T1[i]□T2[i];	Const int N = 50; int T1 [N], T2[N]; for (i = 0 ; i<N ; i++) T1[i]=T2[i];

USUAL VECTOR OPERATIONS

❖ The usual operations on **vectors** (or one-dimensional arrays) include :

3. Research

- Find an element in the vector (exact value or specific criterion (minimum, maximum,)).
- Find the number of occurrences of an element in the vector.

4. Sorting

- Sort the vector in ascending or descending order, with algorithms such as sorting by insertion, by selection, by bubbles, etc.

USUAL VECTOR OPERATIONS

❖ The usual operations on **vectors** (or one-dimensional arrays) include :

5. Insert (add) Elements

- Add an element at the end (an operation often called "push back").
- Insert an element at a given position, which may require an offset of the following elements.

6. Remove (delete) Elements

- Delete an item at a given position, shifting the remaining items.
- Delete the last item (often called "pop back").

USUAL VECTOR OPERATIONS

❖ The usual operations on **vectors** (or one-dimensional arrays) include :

7. Mathematical operations

- Calculation of the sum, the product, or the average of the elements.
- Normalization (e.g., dividing each item by the total sum).
- Element-by-element operations, such as adding a constant to all elements.

8. concatenation

- Combine two vectors to create a new one.

USUAL VECTOR OPERATIONS

❖ The usual operations on **vectors** (or one-dimensional arrays) include :

9. Rotation and Offset

- Shift items to the left or right.
- Circular rotation (the last item returns to the beginning, or vice versa).

10. Filtering.

- Create a sub-vector from a criterion (for example, keep even elements only).

11. Etc

STRINGS

- ❖ A **string** is a sequence (or suites) of characters, usually seen as a sentence or words.
- ❖ It is written in quotation marks (double quote), e.g.: "HELLO", "Give the value of X".
- ❖ It is considered a structured type (an array of characters): a one-dimensional array (vector) of which each element contains a single character.
 - Access to an element (character) is identical to that of vectors.
- ❖ A string variable can be declared in two different ways: a character vector or an abstract type called **String**.

STRINGS

	Character vector	String
Declaration	ch: array [100] of character	ch: string [100]
Reading	Do print("Give n between 1 et 100"); scan(n); while (n<0) or (n>100); For i□0 to n-1 do Begin print("Give a ieme charater"); scan (ch[i]); End _{For} ;	scan(ch); <i>/*the real length "n" is calculated because the string ends with a special null character '\0' (pronounced "backslash zero" or "null terminator") that indicates the end of the string.</i> <i>n□0</i> <i>While (ch[n] != '\0') Do</i> <i>n++</i> <i>*/</i>

STRINGS

	Character vector	String											
Declaration	ch: array [100] of char	ch: string [100]											
Writing	For i□0 to n-1 do print (ch[i]);												
		print (ch)											
Example1 "Hello"	//Reading n□5; ch[0] = 'H'; ch[1] = 'e'; ch[2] = 'l'; ch[3] = 'l'; ch[4] = 'o'; <table><tr><td>H</td><td>e</td><td>l</td><td>l</td><td>o</td></tr></table> //printing Hello	H	e	l	l	o	//Reading ch □ "Hello » <table><tr><td>H</td><td>e</td><td>l</td><td>l</td><td>o</td><td>\0</td></tr></table> //printing Hello	H	e	l	l	o	\0
H	e	l	l	o									
H	e	l	l	o	\0								

STRINGS

❖ Assignments and comparisons are only done by for a single character.

❖ Example

1. `p,q: char ; /* p and q are characters */`
 2. `chaine1: array [10] of char; chaine2: string`
 3. `p □ 'A' ; /* valid instruction */`
 4. `chaine1 □ "Bonjour" ; /* INVALID instruction (A) */`
 5. `chaine2 □ "Hello" ; /* valid instruction */`
 6. `if (p = q) ; /* valid instruction */`
 7. `if (chaine1 = chaine2) /* INVALID instruction (B) */`
- To make the assignment (A) and the comparison (B), it would therefore be necessary to proceed character by character.

STRINGS

- ❑ In C language, a string is a vector of characters that can be declared in different ways: **char ch[Smax]**, **char ch []** or **char ***.
- ❑ There is no abstract type of **string**, but there is a library **<string.h>** that contains several modules for manipulating strings.
- ❑ In this course (or even the PWs of this semester), we will use only the declaration **char ch[Smax]** (where Smax is a constant), and we will return to the other notions (dynamic declaration or declaration as much as pointer and the modules of the **<string.h>** library) in semester 2.

STRINGS

- ❖ The `scanf` and `printf` formatted input-output functions can read and write strings with the format `%s`.
- ❖ The `scanf` function reads on the standard input a sequence of characters up to (and this is its main disadvantage) the encounter of a space spacing character, tab or line break, arranges it at the indicated address and completes it with a `\0` symbol.
- ❖ **For example:**

```
char s[20];  
scanf("%s",&s);  
printf("s = %s", s);
```

Si l'utilisateur saisi au clavier : `Bonjour`

il obtiendra l'affichage :

`s = Bonjour`

STRINGS

- ❖ The `scanf` and `printf` formatted input-output functions can read and write strings with the format `%s`.
- ❖ The `scanf` function reads on the standard input a sequence of characters up to (and this is its main disadvantage) the encounter of a space spacing character, tab or line break, arranges it at the indicated address and completes it with a `\0` symbol.
- ❖ **For example:**

```
char s[20];  
scanf("%s",&s);  
printf("s = %s", s);
```

```
Si maintenant l'utilisateur saisi au clavier : Bonne journée  
il obtiendra l'affichage :  
s = Bonne
```

STRINGS

- ❖ The spacing character between **Bonne** and **journée** ends the reading. However, it is common to have to read a line, that is to say a string of characters containing several words spaced by spaces or tabs and ending with a line break character `\n`.
- ❖ To read a full line, you can use the `gets` function, the prototype of which is as follows:

`char *gets(char *s);`

- ❖ This function reads a string of characters from the standard input (keyboard) until it encounters a newline character (`\n`) or the end of file (EOF). A null character (`\0`) is added at the end of the string.

STRINGS

`char *gets(char *s);`

❖ This function reads a string of characters from the standard input (keyboard) until it encounters a newline character (`\n`) or the end of file (EOF). A null character (`\0`) is added at the end of the string.

❖ Fo

```
char s[20]
```

```
gets(s)
```

```
printf("s = %s",s);
```

Typing "**Bonne journée**" on the keyboard will correctly result in the output:

s = Bonne journée

MATRIXES

- ❖ Matrixes seen in mathematics frequently appear in algorithms and programming, because they constitute a very natural format for storing data (images, screens, animations, databases, game boards, etc.).
- ❖ In a mathematical context, a matrix is an array with n rows and m columns ($n, m \in \mathbb{N}$) that contains numbers (integers, real, complex, ...).
- ❖ In an algorithmic context, a matrix is a two-dimensional array that contains elements of simple (real, integer, character, Boolean) or structured (array, string, record, set) type.

	0	1	2	3
0	3	6	7	1
1	2	5	6	2
2	3	1	4	2

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

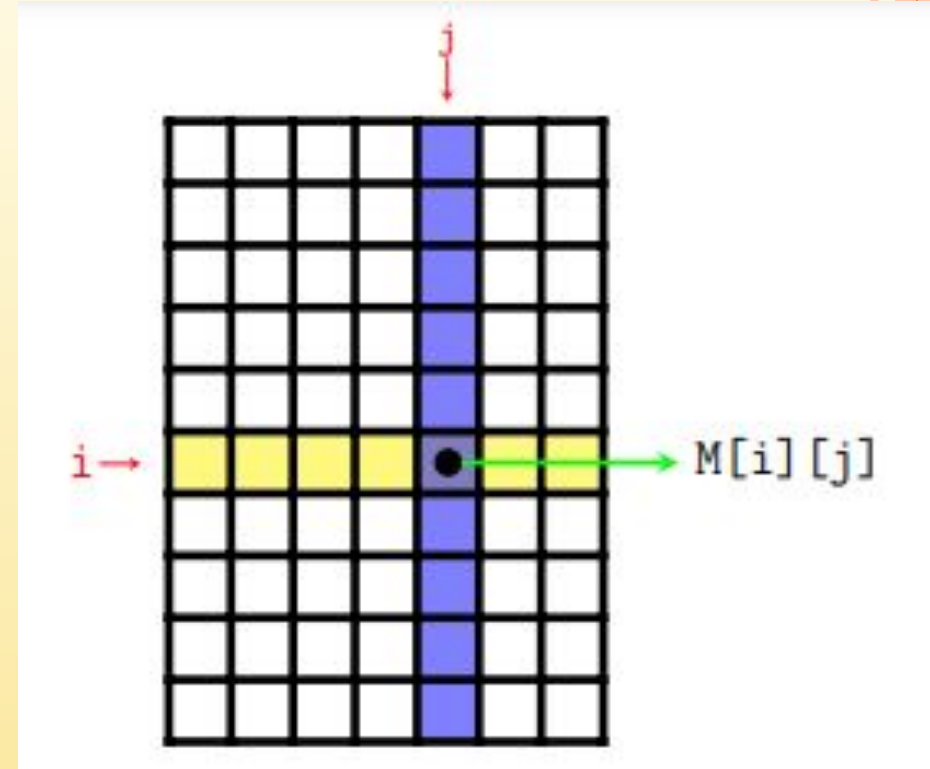
MATRIXES

❖ A matrix is declared as follows

Identifier: array $[S_{\text{maxLig}}][S_{\text{maxCol}}]$ of type;

Where,

- ❑ Identifier: Represents the name of the matrix.
 - ❑ S_{maxLig} : is the maximum number of rows.
 - ❑ S_{maxCol} : is the maximum number of columns.
 - ❑ Type: is the type of single or compound elements.
- ❖ To access the elements of a matrix, we use the operator $[]$ by specifying a position by dimension: we first specify the row number, and then the column number:



MATRIXES

- ❖ Several operations can be performed on the matrices:

Consulting Operations	Modification operations
• Displaying a matrix • Calculation of the sum, the product, the maximum, the minimum, the transpose, the inverse,... for the elements of the matrix • Check the properties of a matrix: diagonal, symmetrical, magic, etc.	• Reading (entering or filling in) a matrix • Edit elements • Items offset

- ❖ All these operations require the elements of the matrix to be traversed. There are several types of paths: row by row, column by column, partial, a row /column, the main diagonal/secondary diagonal, etc.

MATRIXES

- ❖ **Exercise 1:** Let a matrix A of size $n*m$ ($n \leq 20$, $m \leq 30$), write an algorithm that:
1. Reads (fills in) the values of matrix A with two different methods (by row and by column).
 2. Displays the maximum as well as the average of all elements of A .
 3. Constructs matrix B which represents the transpose of matrix A .
 4. Displays the sum of each column of B .
 5. Displays matrix A row by row, and matrix B column by column.

MATRIXES

❖ Exercise 1 Solution:

Algorithm Operations_Matrice1;

Var

A : array of [20] [30] of integer; B: array of [30] [20] of integer;

i, j, n, m, S, Max : integer; Moy : real;

// 1. Read Matrix A

Do

print("give n between 1 et 20 ");

scan(n);

while (n<1) or (n>20);

Do

print("give m between 1 et 30 ");

scan(n);

while (n<1) or (n>30);

MATRIXES

❖ Exercise 1 Solution:

// 1. Read Matrix A

//reading by row

For i $\square$ 0 to n-1 do

 For j $\square$ 1 to m-1 de

 Begin

 print("give an element");

 scan(A[i][j]);

 End_{For};

//reading by column

For i $\square$ 0 to m-1 do

 For j $\square$ 1 to n-1 de

 Begin

 print("give an element");

 scan(A[j][i]);

 End_{For};

MATRIXES

❖ Exercise 1 Solution:

// 2. Display the maximum as well as the average of all the elements of A

```
S ← 0; Max ← A[0][0];
```

```
For i ← 0 to n-1 do
```

```
    For j ← 0 to m-1 de
```

```
        Begin
```

```
            S ← S + A[i][j];
```

```
            if(A[i][j] > Max) then Max ← A[i][j];
```

```
        EndFor
```

```
Moy ← S / (n * m);
```

```
Print("Moy=", Moy, " Max=", Max);
```

MATRIXES

❖ Exercise 1 Solution:

// 3. Construct matrix B which represents the transpose of matrix A.

For i $\square$ 0 to n-1 do

 For j $\square$ 0 to m-1 de

 B[j][i] $\square$ A[i] [j];

// 4. Display the sum of each column of B.

For i $\square$ 0 to m-1 do

 Begin

 S $\square$ 0;

 For j $\square$ 0 to n-1 de

 S $\square$ S+ B[j] [i];

 Print("la somme des éléments de la colonne", i, " de la matrice B est :", S);

End_{For}

MATRIXES

❖ Exercise 1 Solution:

// 5. Display matrix A row by row, and matrix B column by column.

```
//Display by row
```

```
For i □ 0 to n-1 do
```

```
    For j □ 1 to m-1 de
```

```
        print("A[“, i, “] [“, j, “]=“, A[i][j]);
```

```
//Display by column
```

```
For i □ 0 to m-1 do
```

```
    For j □ 1 to n-1 de
```

```
        print("B[“, i, “] [“, j, “]=“, B[i][j]);
```

COURSE SOURCES

- B. CHERGOU (2014), N. MEDJAOUI (2018) & N. BERREHOUMA, *Cours d'algorithmique/pascal*, 1st-year CPI, Higher National School of Computer Science (ESI) Oued Smar, Algiers.
- M. ZAIR (2022), *Algorithmique 1*, 1st-year License Common Core MI (Mathematics and Computer Science), Faculty of Sciences, Saad Dahlab University, Blida.
- O. Hadj Yahia (2022), *Bureautique, algorithmique et langage de programmation*, 1st-year License Common Core SM, Faculty of Sciences, Saad Dahlab University, Blida.
- M. Le Gonidec, INTRODUCTION A LA PROGRAMMATION– PRATIQUE DU LANGAGE C –University of SUD, Disponible:
<https://www.cours-gratuit.com/cours-c/introduction-a-la-programmation-pratique-du-langage-c-et-c>



Université Saad Dahlab – Blida 1

Faculté des Sciences

Département d'Informatique

Ingénieur en Informatique

Semestre 1 (1^{ère} année)

Algorithmique et Structure de Données 1



CHAPITRE III: TABLEAUX

Dr. MUSTAPHA CHABA MOUNA(chabamouna_mustapha@univ-blida.dz)

2025-2026

CONTENU

- ❑ Introduction
- ❑ Définitions
- ❑ Vecteurs.
- ❑ Chaines de Caractères.
- ❑ Matrices.

INTRODUCTION

- ❖ Certaines situation ne peuvent pas être traitées avec des variables simples uniquement. **Elles ne peuvent garder qu'une seule valeur à la fois.**
- ❖ Utiliser des données multiples plusieurs fois peut être délicat dans un programme.
 - Exemple: Classer les étudiants du groupe (30 étudiants!) selon leur moyenne □ utiliser 30 variables réelles!!

```
Var M1 , M2 ,M3 ,M4 ,M5 ,M6 ,M7 ,M8 ,M9 ,M10 ,M11 ,M12 ,M13 ,.....: real
```

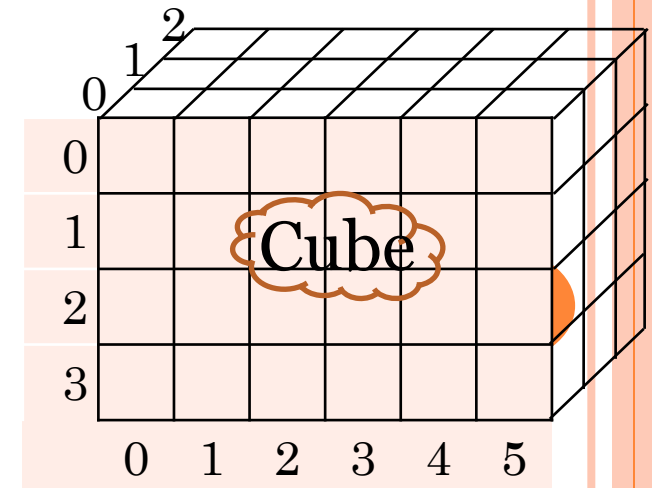
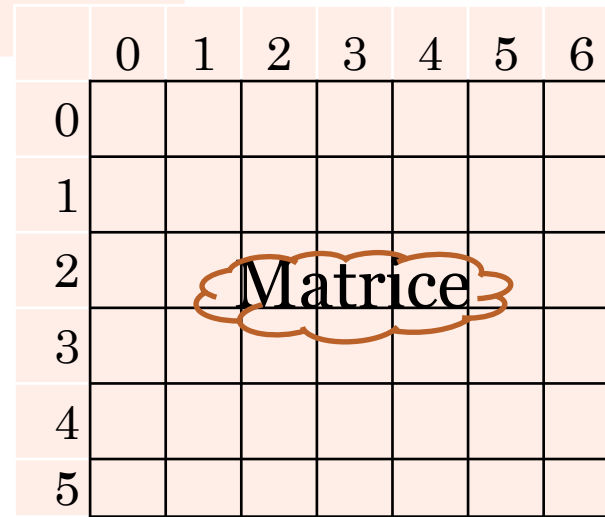
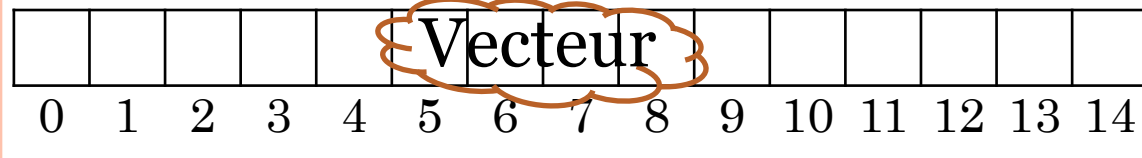
- **Tableaux: permettent de regrouper les moyennes dans une structures.**

M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12	...	...	M30
----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----

DÉFINITIONS

- ❖ Les **tableaux (arrays)** sont des variables structurées, constituées d'un nombre fixe (statique) d'éléments, tous de même type, et accessibles grâce à des indices. Ils sont caractérisés par leur dimension:

Dimension	Appellation
1	Tableau unidimensionnel ou Vecteur
2	Tableau bidimensionnel ou Matrice
3	Tableau tridimensionnel ou Cube
N>3	Tableau multidimensionnel



DÉFINITIONS

- ❖ Pour définir un tableau, il faut spécifier:
 - sa dimension
 - le nombre maximal de ses éléments dans chaque dimension (soit d), c'est à dire sa taille maximale dans chaque dimension (soit $Tmax_{dimension}$)
 - le type de chaque élément (*typelt*)
 - son nom (*identificateur*)
- ❖ La déclaration se fait en précisant le mot **ARRAY**, suivi de la taille entre crochets de chaque dimension suivi du mot **OF** et du type des éléments.

identificateur : **ARRAY** [$Tmax_1$] *[/i> [$Tmax_2$] [$Tmax_d$] */* **OF** *typelt**

Le nombre des crochets [] permet de désigner la **dimension (d)** du tableau

DÉFINITIONS

❖ Exemple:

Dimension	Langage Algorithmique (LA)	Langage C (LC)
1: <i>Vecteur</i>	VAR V: array [100] of integer	int V [100];
2: <i>Matrice</i>	VAR M: array [100][200] of integer	int M [100][200];
3: <i>Cube</i>	VAR C: array [100][200][300] of integer	int C [100][200][300];

- V est un vecteur de 100 éléments,
- M est une matrice de 100 lignes et 200 colonnes (chaque ligne est un vecteur)
- C est un cube de 100 couches où chaque couche ayant 200 lignes et 300 colonnes (chaque couche est une matrice).

DÉFINITIONS

❖ D'autres méthodes de déclaration du tableau existent:

N° Méthode	Langage Algorithmique (LA)	Langage C (LC)
1: déclarer autant que variable	VAR A: array [100] of integer	int A [100];
2: utiliser une constante pour déclarer la taille maximale (Tmax)	CONST Tmax1 = 100 VAR A: array [Tmax] of integer	const int Tmax = 100; int A [Tmax];
3: déclarer autant qu'un nouvel type	CONST Tmax = 100 TYPE TAB:array [Tmax] of integer VAR A : TAB	const int Tmax = 100; typedef int TAB [Tmax]; TAB A;

DÉFINITIONS

- ❖ Dans les tableaux statiques, on distingue souvent entre la taille maximale (ou la taille déclarée) et la taille réelle (ou la taille utilisée) du tableau.

	Taille maximale (déclarée)	Taille réelle (utilisée)
Définition	Elle représente la capacité totale du tableau, c'est-à-dire le nombre maximum d'éléments que le tableau peut stocker.	Elle représente le nombre d'éléments réellement utilisés dans le tableau à un moment donné.
Déclaration et modification	Elle est déterminée au moment de la déclaration et reste fixe pendant toute la durée de vie du tableau.	Elle est déterminée au cours de l'exécution (en général par une lecture) et peut varier en fonction de la logique applicative.
Remarques	Il ne faut pas donner une taille trop grande pour ne pas gaspiller de mémoire.	Elle doit être inférieure ou égale à la taille maximale du tableau.

DÉFINITIONS

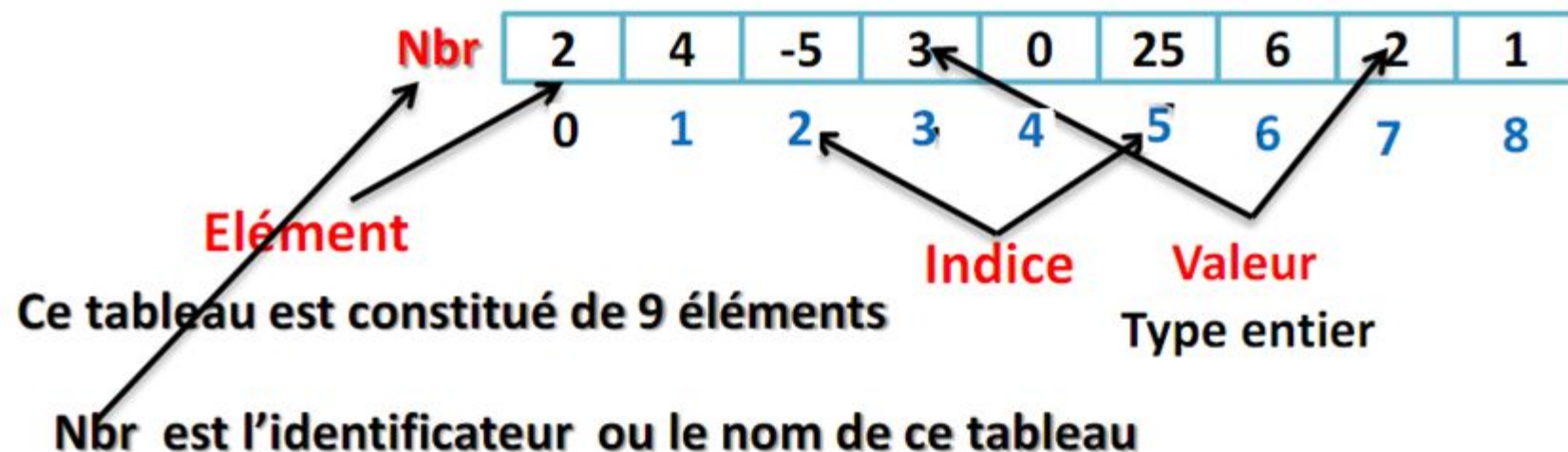
- ❖ L'accès à un élément du tableau s'effectue en précisant le nom du tableau suivi de la valeur de l'indice entre crochets pour chaque dimension:

identificateur [**indice**₁] *[[indice*₂*].... [indice*₃*]]*

- avec **indice**_d : une constante, une variable ou une expression entière dont la valeur doit être entre 0 et (**Tmax**_d - 1).

- ❖ **EXEMPLE 1 (Vecteur):**

- Nbr: array [9] of integer
- Le troisième élément est à la position 2.



DÉFINITIONS

- ❖ **EXEMPLE 1 (Vector):** Soit un tableau COEF contenant les coefficients de 7 matières

6	4	5	3	2	2	1
0	1	2	3	4	5	6

- **COEF: array [7] of integer**
- On peut écrire les actions suivantes
 - $A \leftarrow \text{COEF}[3]$; $B \leftarrow \text{COEF}[k]$; $C \leftarrow \text{COEF}[(\text{test}+k) \bmod i]$
 - $\text{COEF}[6] \leftarrow E$; $\text{COEF}[m] \leftarrow F$; $\text{COEF}[a * \text{res} \bmod 2] \leftarrow G$
- On constate qu'un élément du tableau est considéré comme une variable et que la valeur de l'indice peut être une constante, une variable ou une expression dont la valeur doit être entre 0 et 6 ($T_{\max}-1$).

OPERATIONS USUELLES SUR LES VECTEURS

- ❖ Les opérations usuelles sur les **vecteurs** (ou tableaux à une seule dimension) comprennent :

1. Initialisation d'un vecteur

- Initialisation vide : créer un vecteur sans valeurs initiales.
- Initialisation avec des valeurs données, par exemple, des valeurs lues du clavier (**lecture des éléments d'un vecteur**).
- Initialisation avec une valeur par défaut (zéros ou autres).

OPERATIONS USUELLES SUR LES VECTEURS

- ❖ **Note 1 (initialisation):** Dans le langage C, les tableaux statiques peuvent être **initialisés lors de leur déclaration**. L'initialisation peut se faire en spécifiant les valeurs entre les crochets { } lors de la déclaration. Si les valeurs ne sont pas spécifiées, les éléments du tableau sont initialisés avec des valeurs par défaut (0 pour les types numériques).

Déclaration	Image du tableau																
int T1[3] = {8, 1, 12};	<table><tr><td>8</td><td>1</td><td>12</td></tr></table>	8	1	12													
8	1	12															
int T2[8] = {1, 5, 1}																	
int T3[] = {10, 5, 8, -1};	<table><tr><td>1</td><td>5</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>10</td><td>5</td><td>8</td><td>-1</td><td></td><td></td><td></td><td></td></tr></table>	1	5	1	0	0	0	0	0	10	5	8	-1				
1	5	1	0	0	0	0	0										
10	5	8	-1														

OPERATIONS USUELLES SUR LES VECTEURS

❖ Note 1 (initialisation):

- Il est toujours possible d'initialiser un tableau en utilisant une boucle dans le corps du programme principal ou module.
- Par exemple, on peut initialiser le tableau T de différentes manières :

Initialisation	Langage Algorithmique (LA) T : array [20] of integer	Langage C (LC) int T [20];
avec une seule constante (c)	for i□0 to 19 do T[i] □ c;	for (i = 0 ; i<20 ; i++) T[i]=c;
avec des constantes différentes	for i□0 to 19 do T[i] □ i;	for (i = 0 ; i<20 ; i++) T[i]=i;
Avec des valeurs lues	for i□0 to 19 do scan(T[i]);	for (i = 0 ; i<20 ; i++) scanf("%d", &T[i]);

OPERATIONS USUELLES SUR LES VECTEURS

- ❖ Les opérations usuelles sur les **vecteurs** (ou tableaux à une seule dimension) comprennent :

1. Initialisation d'un vecteur

- Initialisation vide : créer un vecteur sans valeurs initiales.
- Initialisation avec des valeurs données, par exemple, des valeurs lues du clavier (**lecture des éléments d'un vecteur**).
- Initialisation avec une valeur par défaut (zéros ou autres).

2. Parcours

- Parcourir le vecteur pour **afficher**, modifier ou effectuer des calculs sur les éléments.
- Copier un vecteur dans un autre.

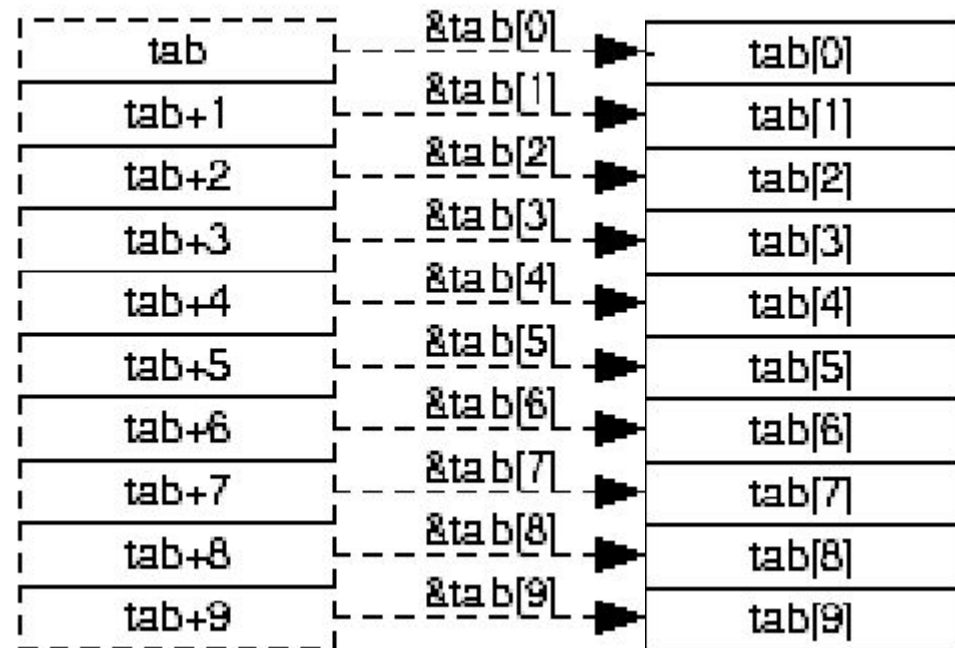
OPERATIONS USUELLES SUR LES VECTEURS

❖ Note 2 (Affectation d'un tableau à un autre)

- La déclaration d'un tableau réserve de la place en mémoire pour les éléments du tableau et fournit l'adresse du premier élément du tableau dans le nom du tableau (sans crochets).

□ Exemple:

tab: array[10] of integer



OPERATIONS USUELLES SUR LES VECTEURS

❖ Note 2 (Affectation d'un tableau à un autre)

- Le nom du tableau correspond en fait à l'adresse du premier élément du tableau. Cela implique en particulier qu'aucune opération globale n'est autorisée sur un tableau. **Notamment, un tableau ne peut pas figurer à gauche d'un opérateur d'affectation.**
- Par exemple, **on ne peut pas écrire “ $T1 \square T2$ ”**. Il faut effectuer l'affectation pour chacun des éléments du tableau en utilisant une boucle:

Copier un tableau dans un autre	Langage Algorithmique (LA)	Langage C (LC)
	Const N = 50 T1, T2 : array [N] of integer for i $\square$ 0 to N-1 do T1[i] $\square$ T2[i];	Const int N = 50; int T1 [N], T2[N]; for (i = 0 ; i < N ; i++) T1[i]=T2[i];

OPERATIONS USUELLES SUR LES VECTEURS

- ❖ Les opérations usuelles sur les **vecteurs** (ou tableaux à une seule dimension) comprennent :

3. Recherche

- Trouver un élément dans le vecteur (valeur exacte ou critère spécifique (minimum, maximum,)).
- Trouver le nombre d'occurrence d'un élément dans le vecteur.

4. Tri

- Trier le vecteur en ordre croissant ou décroissant, avec des algorithmes comme le tri par insertion, par sélection, par bulles,

OPERATIONS USUELLES SUR LES VECTEURS

- ❖ Les opérations usuelles sur les **vecteurs** (ou tableaux à une seule dimension) comprennent :

5. Insertion d'éléments

- Ajouter un élément à la fin (opération souvent appelée "push back").
- Insérer un élément à une position donnée, ce qui peut nécessiter un décalage des éléments suivants.

6. Suppression d'éléments

- Supprimer un élément à une position donnée, avec décalage des éléments restants.
- Supprimer le dernier élément (souvent appelé "pop back").

OPERATIONS USUELLES SUR LES VECTEURS

- ❖ Les opérations usuelles sur les **vecteurs** (ou tableaux à une seule dimension) comprennent :

7. Opérations mathématiques

- Calcul de la somme, du produit, ou de la moyenne des éléments.
- Normalisation (par exemple, division de chaque élément par la somme totale).
- Opérations élément par élément, comme l'ajout d'une constante à tous les éléments.

8. Concaténation

- Combiner deux vecteurs pour en créer un nouveau.

OPERATIONS USUELLES SUR LES VECTEURS

- ❖ Les opérations usuelles sur les **vecteurs** (ou tableaux à une seule dimension) comprennent :

9. Rotation et décalage

- Décaler les éléments vers la gauche ou la droite.
- Rotation circulaire (le dernier élément revient au début, ou vice-versa).

10. Filtrage

- Créer un sous-vecteur à partir d'un critère (par exemple, garder les éléments pairs uniquement).

11. etc

CHAINE DE CARACTÈRES

- ❖ Une chaîne de caractères (**string** en anglais) une séquence (ou suite) de caractères, généralement vue comme étant une phrase ou des mots.
- ❖ Elle s'écrit entre guillemets (double quote), e.g: "Bonjour", "HELLO", "Donner la valeur de X".
- ❖ Elle est considéré comme un type structuré (un tableau de caractères): un tableau à une dimension (vecteur) dont chaque élément contient un seul caractère. L'accès à un élément (caractère) est identique à celui des vecteurs.
- ❖ Une variable de type chaîne de caractères peut être déclarée de deux manières différentes: un vecteur de caractère ou un type abstrait appelé **String**.

CHAINE DE CARACTÈRES

	Vecteur de caractères	String
Déclaration	ch: array [100] of character	ch: string [100]
Lecture	<pre> Do print("Donner n entre 1 et 100"); scan(n); while (n<0) or (n>100); For i□0 to n-1 do Begin print("Donner le ieme caractère"); scan (ch[i]); End_{For}; </pre>	<pre> scan(ch); <i>/*la longueur réelle « n » est calculée car la chaîne se terminée par un caractère spécial nul '\0' (prononcé "backslash zéro" ou "null terminator") qui indique la fin de la chaîne. n□0 While (ch[n] != '\0') Do n++ */</i> </pre>

CHAINE DE CARACTÈRES

	Vecteur de caractères	String											
Déclaration	ch: array [100] of char	ch: string [100]											
Ecriture	For i□0 to n-1 do print (ch[i]);												
		print (ch)											
Exemple1 "Hello"	//Lecture n□5; ch[0] = 'H'; ch[1] = 'e'; ch[2] = 'l'; ch[3] = 'l'; ch[4] = 'o'; <table><tr><td>H</td><td>e</td><td>l</td><td>l</td><td>o</td></tr></table> //Affichage Hello	H	e	l	l	o	//Lecture ch □ "Hello » <table><tr><td>H</td><td>e</td><td>l</td><td>l</td><td>o</td><td>\0</td></tr></table> //Affichage Hello	H	e	l	l	o	\0
H	e	l	l	o									
H	e	l	l	o	\0								

CHAINE DE CARACTÈRES

❖ Les affectations et Les comparaisons ne se font que par pour 1 seul caractère.

❖ Exemple

1. `p,q: char ; /* p et q sont des caractères */`
 2. `chaine1: array [10] of char; chaine2: string`
 3. `p □ 'A' ; /* instruction valide */`
 4. `chaine1 □ "Bonjour" ; /* instruction NON valide (A) */`
 5. `chaine2 □ "Hello" ; /* instruction valide */`
 6. `if (p = q) ; /* instruction valide */`
 7. `if (chaine1 = chaine2) /* instruction NON valide (B) */`
- Pour faire l'affectation (A) et la comparaison (B), il faudrait donc procéder caractère par caractère.

CHAÎNE DE CARACTÈRES

- En langage C, une chaîne de caractère est un vecteur de caractères qui pourra être déclaré de différentes manières: **char ch[Smax]**, **char ch []** ou **char ***.
- Il n'y a pas le type abstrait **string**, mais il existe une bibliothèque **<string.h>** qui contient plusieurs modules permettant de manipuler les chaînes de caractères.
- Dans ce cours (voire les TP de ce semestre), nous allons utiliser seulement la déclaration **char ch[Smax]** (où **Smax** est une constante), et nous allons revenir sur les autres notions (déclaration dynamique ou déclaration autant que pointeur et les modules de la bibliothèques **<string.h>**) dans le semestre 2.

CHAINE DE CARACTÈRES

- ❖ Les fonctions d'entrée-sortie formatées `scanf` et `printf` peuvent lire et écrire des chaînes de caractères avec le format `%s`.
- ❖ La fonction `scanf` lit sur l'entrée standard une suite de caractères jusqu'à (et c'est là son principal inconvénient) la rencontre d'un caractère d'espacement espace, tabulation ou saut de ligne, la range à l'adresse indiquée et la complète par un symbole `\0`.
- ❖ **Exemple:**

```
char s[20];  
scanf("%s",&s);  
printf("s = %s", s);
```

Si l'utilisateur saisi au clavier : `Bonjour`

il obtiendra l'affichage :

`s = Bonjour`

CHAINE DE CARACTÈRES

- ❖ Les fonctions d'entrée-sortie formatées `scanf` et `printf` peuvent lire et écrire des chaînes de caractères avec le format `%s`.
- ❖ La fonction `scanf` lit sur l'entrée standard une suite de caractères jusqu'à (et c'est là son principal inconvénient) la rencontre d'un caractère d'espacement espace, tabulation ou saut de ligne, la range à l'adresse indiquée et la complète par un symbole `\0`.
- ❖ Exemple:

```
char s[20];  
scanf("%s",&s);  
printf("s = %s", s);
```

Si maintenant l'utilisateur saisi au clavier : Bonne journée
il obtiendra l'affichage :

s = Bonne

CHAINE DE CARACTÈRES

- ❖ Le caractère d'espacement entre **Bonne** et **journée** met fin à la lecture. Or il est fréquent d'avoir à lire une ligne, c'est-à-dire une chaîne de caractères contenant plusieurs mots espacés par des espaces ou des tabulations et se terminant par un caractère de saut de ligne `\n`.
- ❖ Pour lire une ligne complète, on peut utiliser la fonction `gets`, dont le prototype est le suivant:

`char *gets(char *s);`

- ❖ Cette fonction permet de lire une chaîne de caractères depuis l'entrée standard (clavier) jusqu'à ce qu'elle rencontre un saut de ligne (`\n`) ou la fin de fichier (EOF). Un caractère nul `\0` est ajouté en fin de chaîne.

CHAINE DE CARACTÈRES

`char *gets(char *s);`

❖ Cette fonction permet de lire une chaîne de caractères depuis l'entrée standard (clavier) jusqu'à ce qu'elle rencontre un saut de ligne (`\n`) ou la fin de fichier (EOF). Un caractère nul `\0` est ajouté en fin de chaîne.

❖ Exemple:

```
char s[20]
```

```
gets(s)
```

```
printf("s = %s",s);
```

La saisie au clavier de **Bonne journée** provoquera bien l'affichage

s = Bonne journée

MATRICES

- ❖ Les matrices vues en mathématiques apparaissent fréquemment en algorithmique et en programmation, car elles constituent un format très naturel pour stocker les données (images, écrans, animations, bases de données, plateaux de jeu,...).
- ❖ Dans un contexte mathématique, une matrice est un tableau à n lignes et m colonnes ($n, m \in \mathbb{N}$) qui contient des nombres (entiers, réels, complexes, ...).
- ❖ Dans un contexte algorithmique, une matrice est un tableau à deux dimension qui contient des éléments de type simple (réel, entier, caractère, booléen) ou structuré (tableau, chaîne de caractère, enregistrement, ensemble).

	0	1	2	3
0	3	6	7	1
1	2	5	6	2
2	3	1	4	2

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

MATRICES

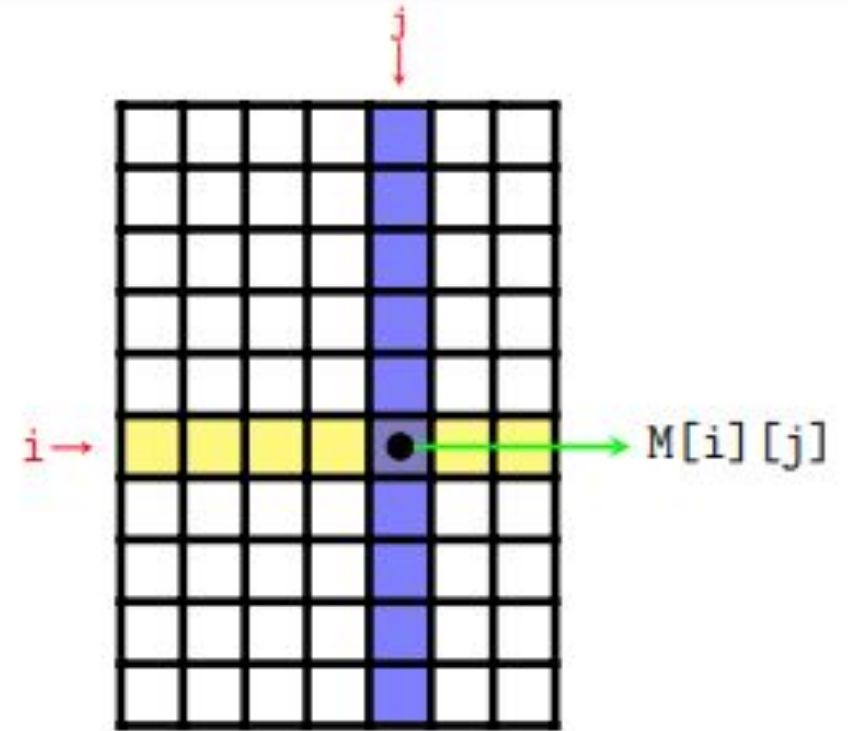
❖ Une matrice est déclarée comme suit

Identificateur : array [S_{maxLig}][S_{maxCol}] of type;

Tel que :

- Identificateur : représente le nom de la matrice.
- S_{maxLig} : est le nombre maximal de lignes.
- S_{maxCol} : est le nombre maximal de colonnes.
- Type : est le type des éléments simple ou composé.

❖ Pour accéder aux éléments d'une matrice, on utilise l'opérateur [] en spécifiant une position par dimension: on précise d'abord le numéro de ligne, et puis le numéro de colonne :



MATRICES

- ❖ Plusieurs opérations peuvent être effectuées sur les matrices :

Opérations de consultation	Opérations de modification
• Affichage d'une matrice • Calcul de la somme, le produit, le maximum, le minimum, le transposée, l'inverse, ... pour les éléments de la matrice • Vérifier les propriétés d'une matrice: diagonale, symétrique, magique,	• Lecture (saisie ou remplissage) d'une matrice • Modification des valeurs des éléments • Décalage ou permutation des éléments

- ❖ Toutes ces opérations nécessitent le parcours des éléments de la matrice. On peut citer plusieurs types de parcours : ligne par ligne, colonne par colonne, partiel, une ligne / une colonne, la diagonale principale / diagonale secondaire,

MATRICES

- ❖ **Exercice 1** : Soit une matrice A de taille $n*m$ ($n \leq 20$, $m \leq 30$), écrire un algorithme qui :
1. Lit (remplit) les valeurs de la matrice A avec deux méthodes différentes (par ligne et par colonne).
 2. Affiche le maximum ainsi que la moyenne de tous les éléments de A .
 3. Construit la matrice B qui représente la transposée de la matrice A .
 4. Affiche la somme de chaque colonne de B .
 5. Affiche la matrice A ligne par ligne, et la matrice B colonne par colonne.

MATRICES

❖ Solution Exercice 1 :

Algorithm Operations_Matrice1;

Var

A : array of [20] [30] of integer; B: array of [30] [20] of integer;

i, j, n, m, S, Max : integer; Moy : real;

// 1. Lire la matrice A

Do

print("give n between 1 et 20 ");

scan(n);

while (n<1) or (n>20);

Do

print("give m between 1 et 30 ");

scan(n);

while (n<1) or (n>30);

MATRICES

❖ Solution Exercice 1 :

// 1. Lire la matrice A

//lecture par ligne

For i $\square$ 0 to n-1 do

 For j $\square$ 1 to m-1 de

 Begin

 print(“give an element”);

 scan(A[i][j]);

 End_{For};

//lecture par colonne

For i $\square$ 0 to m-1 do

 For j $\square$ 1 to n-1 de

 Begin

 print(“give an element”);

 scan(A[j][i]);

 End_{For};

MATRICES

❖ Solution Exercice 1 :

// 2. Afficher le maximum ainsi que la moyenne de tous les éléments de A

```
S ← 0;  Max ← A[0][0];
```

```
For i ← 0 to n-1 do
```

```
  For j ← 0 to m-1 de
```

```
    Begin
```

```
      S ← S + A[i][j];
```

```
      if(A[i][j] > Max) then  Max ← A[i][j];
```

```
    EndFor
```

```
Moy ← S/(n*m);
```

```
Print("Moy=", Moy, " Max=", Max);
```

MATRICES

❖ Solution Exercice 1 :

// 3. Construire la matrice B qui représente la transposée de la matrice A.

For i $\square$ 0 to n-1 do

 For j $\square$ 0 to m-1 de

 B[j][i] $\square$ A[i] [j];

// 4. Afficher la somme de chaque colonne de B.

For i $\square$ 0 to m-1 do

 Begin

 S $\square$ 0;

 For j $\square$ 0 to n-1 de

 S $\square$ S+ B[j] [i];

 Print("la somme des éléments de la colonne", i, " de la matrice B est :", S);

End_{For}

MATRICES

❖ Solution Exercice 1 :

// 5. Afficher la matrice A ligne par ligne, et la matrice B colonne par colonne.

```
//Affichage par ligne
```

```
For i □ 0 to n-1 do
```

```
    For j □ 1 to m-1 de
```

```
        print("A[“, i, “] [“, j, “]=“, A[i][j]);
```

```
//Affichage par colonne
```

```
For i □ 0 to m-1 do
```

```
    For j □ 1 to n-1 de
```

```
        print("B[“, i, “] [“, j, “]=“, B[i][j]);
```

SOURCES DE CE COURS

- ❑ B. CHERGOU (2014) & N. MEDJAOUI (2018), Cours d'algorithmique/pascal, 1ère année CPI, Ecole nationale Supérieure d'Informatique (ESI), Oued Smar, Alger.
- ❑ O. Hadj Yahia (2022), Bureautique, algorithmique et langage de programmation, 1^{ère} année Licence, Tronc Commun SM, Faculté des Sciences, Université Saad Dahlab de Blida.
- ❑ M. ZAIR (2022), Algorithmique 1, 1^{ère} année Licence, Tronc Commun MI (Mathématique Informatique), Faculté des Sciences, Université Saad Dahlab de Blida.
- ❑ M. Le Gonidec, INTRODUCTION A LA PROGRAMMATION– PRATIQUE DU LANGAGE C –Université du SUD, Disponible:
<https://www.cours-gratuit.com/cours-c/introduction-a-la-programmation-pratique-du-langage-c-et-c>